COURS DÉVELOPPEMENT PHP ORIENTÉ OBJET

Table des matières

CHAPITRE 1 : Introduction à la POO en PHP 4
I. Généralité 4
A. Présentation du langage PHP 4
1) Historique 4
2) Définition 4
3) L'interprétation du code par le serveur 4
4) Implantation au sein du code html 5
B. Les bases du langage php 5
1) Les commentaires 5
2) Les variables 5
3) les opérateurs 7
4) Les structures conditionnelles 7
5) Les boucles 8
C. Les fonctions 10
1) Déclaration d'une fonction : 10
2) Appel de fonction 10
3) Renvoi d'une valeur par une fonction 11
4) Les arguments d'une fonction 11
6) Retourner plusieurs variables 11
II. Les bases du POO en PHP 12
A. Les notions de la programmation orientée objet 12
1) Le concept d'objet et de classe 12
2) Le principe d'encapsulation 12
3) L'héritage 13
4) Le polymorphisme 13
B. Les classes et objets 13
1) Déclaration d'une classe 13
2) Instanciation de la classe 14
3) Accéder aux propriétés d'un objet 14
4) Accéder aux méthodes d'un objet 14
5) Constructeur et destructeurs 14
6) Visibilité 16
7) Statique 18
8) Constantes de classe 19
CHAPITRE 2 : Concepts avancés de POO 20
I. L'héritage 20
1) Présentation 20
2) L'instruction parent 21
II. Le polymorphisme 21
III. Abstraction de classes 22
IV. Interfaces 24
CHAPITRE 3 : Autoloading, namespaces et traits 26
I. Comment inclure les fichiers en PHP 26
1) Inclure un fichier avec la fonction include 26
2) Inclure des fichiers avec la fonction Require 27
3) Inclusion avec les fonctions require_once et include_once 28
II. Autoloading 28
1) La fonction __autoload 28
2) Fonction spl_autoload_register() 28

3) Créer une classe Autoloader 29
III. Les Namespaces 30
1) Déclaration d'un espace de noms 30
2) Accéder aux éléments d'un espace de noms 31
3) La commande namespace et la constante __NAMESPACE__ 32
4) Importation (des éléments) d'un espace de noms et création d'alias 32
IV. Les traits 33
1) Principe des traits 33
2) Création 33
3) Utilisation des traits 34
CHAPITRE 4 : Gestion des erreurs et exceptions 35
I. Les exceptions 35
1) Déclenchement avec throw 35
2) try ... catch 35
3) classe Exception et classes personnalisées 36
4) Plusieurs Catch à la suite 37
5) Finally : exécuter du code avant une erreur fatale 37
6) PHP : Erreur ou Exception ? 38
7) Exception par défaut 38
CHAPITRE 5 : PHP POO et base de données (PDO) 39
I. Présentation 39
II. L'extension mysqli 39
A. La fonction mysqli 39
1) Se connecter à la base de données 39
2) Les requêtes simples 40
3) Les requêtes préparées 41
Exemples 42
B. Avec la classe mysqli 43
1) Ouvrir une connexion la base de données MySQL 43
2) Fermer la connexion 44
3) Les methodes supplémentaires 44
III. PDO : Interface d'accès aux bases de données 45
1) Présentation 45
2) Avantages de PDO 45
3) Activation - Php Windows 45
4) Connexion : création et gestion d'erreur 46
5) Méthodes : exec et query 47
6) Méthode : prepare 49
7) Sécuriser sans passer par des requêtes préparées 50
IV. ORM et CRUD 50
1) Présentation 50
a) Définition de ORM 50
b) Comment fonctionne l'ORM ? 51
c) Avantages de l'utilisation d'un ORM 51
d) Inconvénients de l'ORM 51
e) Comparaison entre ORM et SQL 51
f) Qu'est-ce que le CRUD ? 51
2) Design Pattern "Active Record" 51
a) Avantages 52
b) Inconvénients 52
c) Utilisation dans les Frameworks 52

3) Design Pattern “Data Mapper” 52
d) Concept et Fonctionnement 52
e) Avantages 53
f) Inconvénients 53
CHAPITRE 6 : Architecture web et POO 54
I. Séparation des responsabilités (modèle-vue-contrôleur – MVC) 54
II. Le routage simple 54
1) Présentation 54
2) Principe de base 55
3) Avantages 55
III. Traitement des formulaires 55
1) Les parties essentielles d'un formulaire 56
2) La méthode HTTP GET et POST 57
3) Les tableaux superglobaux $_POST et $_GET 57
IV. LES SESSIONS 57
1) Principe Général de Fonctionnement 58
2) PHP Session - Comment fonctionne une session ? 58
Des exemples 58
a) PHP Session - Demarrer une session 58
b) Créer une variable de session 58
d) PHP Session - Utiliser la valeur d’une variable de session 58
d) PHP Session - Supprimer une variable de session 59
e) PHP Session - Détruire toutes les variables de session 59
f) PHP Session - Détruire une session 59
3) LES COOKIES 59

CHAPITRE 1 : Introduction à la POO en PHP

Dans ce chapitre, avant d'aborder la programmation objet, va présenter les généralités du langage PHP et rappeler les notions de bases du langage en mettant l'accent sur les fonctions.

I.Généralité

A.Présentation du langage PHP

1) Historique

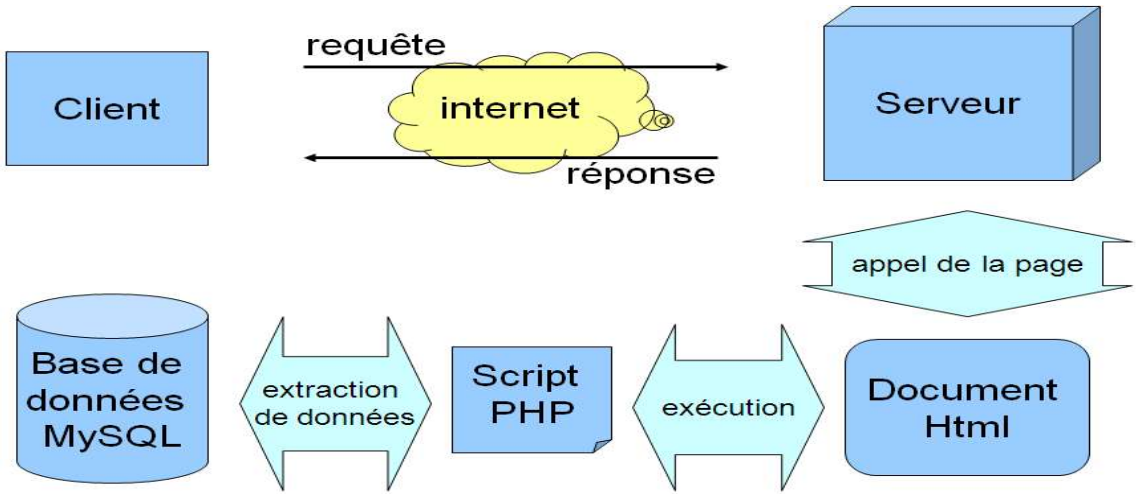
Le langage PHP a été créé en 1994 par Rasmus Lerdorf pour lui permettre de conserver la trace des utilisateurs qui consultaient son CV sur son site. Ainsi en 1995 la première version de ce langage qu'il baptisa *Personal Sommaire Page Tools*, puis *Personal Home Page v1.0* a été présentée. Le PHP va évoluer pour donner des versions dont le PHP 3 en 1998.
En 1997, PHP devient un projet collectif et son interpréteur est réécrit par Zeev Suraski et Andi Gutmans pour donner la version 3 qui s'appelle désormais *PHP : Hypertext Preprocessor*
Les dernières versions du PHP développe la notion d'objet.
PHP permet un interfaçage simple avec de nombreux SGBD (Système de Gestion de Bases de Données) comme : MySQL ,MariaDB, Oracle , PostgreSQL , Solid , Sybase…

2) Définition

Le PHP est un langage de script communiquant directement et exclusivement avec le serveur (comme les scripts CGI, ASP, ...) et non pas avec un client (un script écrit en Javascript ou une applet Java).
La syntaxe du langage provient de celles du langage C, du Perl et de Java.
D'autre part il se caractérise par :

- La gratuité et la disponibilité du code source,
- La simplicité,
- La possibilité et facilité d'implémentation dans une page HTML,
- La simplicité d'interfaçage avec des bases de données,
- L'intégration au sein de nombreux serveurs web (Apache, Microsoft IIS, ...).

3) L'interprétation du code par le serveur



Un script PHP est un simple fichier texte contenant des incluses dans un code HTML à l'aide de balises spéciales et stocké sur le serveur.
Ce fichier doit avoir l'extension ".PHP" pour pouvoir être interprété par le serveur.
Ainsi, lorsqu'un navigateur (le client) désire accéder à une page réalisée en PHP:

- le serveur reconnait qu'il s'agit d'un fichier PHP,
- il lit le fichier PHP,
- Dès que le serveur rencontre une balise indiquant que les lignes suivantes sont du code PHP, il passe en mode PHP, ce qui signifie qu'il exécute les instructions,
- Lorsque le serveur rencontre une instruction, il la transmet à l'interpréteur,
- L'interpréteur exécute l'instruction puis envoie les sorties éventuelles au serveur.

Le code PHP stocké sur le serveur n'est donc **jamais** visible par le client.
De cette façon aucune modification n'est à apporter sur les navigateurs...

4) Implantation au sein du code html

Pour que le script soit interprété par le serveur :
Le fichier contenant le code doit avoir l'extension *.PHP* et non *.html*
Le code PHP contenu dans le code HTML doit être délimité par les balises *<?PHP* et *?>*
Pour des raisons de conformité avec certaines normes (XML et ASP par exemple), plusieurs balises peuvent être utilisées pour délimiter un code PHP:

- *<?PHP* et *?>*
- *<?* et *?>*
- *<script language="PHP">* et *</script>*
- *<%PHP* et *%>*

```
<html>
<head>
<title>Test</title>
</head>
<body>
<p>un bout de code en HTML</p>
<?php
echo 'Mon premier script en PHP';
?>
</body>
</html>
```

On notera bien que la fonction ***echo*** permet d'afficher sur le navigateur.

B. Les bases du langage php

1) Les commentaires

Les commentaires en php se font par :

```
<?php
/* commentaire
sur plusieurs
lignes */
// commentaire de fin de ligne
# commentaire de fin de ligne comme en Shell
?>
```

2) Les variables

Les variables en langage PHP peuvent être de trois types:

- scalaires
- tableaux
- tableaux associatifs

Quelque soit le type de variable, son nom doit obligatoirement être précédé du caractère dollar ($).
Contrairement à de nombreux langages de programmation, comme le langage C, une variable en

PHP n'a pas besoin d'être déclarée, son contenu est utilisé, sinon l'interpréteur lui affectera la valeur 0 par défaut.
Les noms de variables doivent répondre à certains critères:

- un nom de variable doit commencer par une lettre ou un "_" (pas par un chiffre)
- un nom de variables peut comporter des lettres, des chiffres et le caractère _

a)Variables scalaires :

Le langage PHP en propose trois types :

- entiers
- réels (*double*)
- chaînes de caractères

Le langage PHP n'exige pas de typer les variables, il suffit, par contre, de leur assigner une valeur pour en définir le type:
entiers: nombre sans virgule
réels: nombres avec une virgule
chaînes de caractères: ensembles de caractères entre guillemets simples ou doubles

Instruction	**Type de la variable**
$Variable = 0;	type entier
$Variable = 12;	type entier
$Variable = 0.0;	type réel
$Variable = 12.0;	type réel
Variable = "Bonjour tout le monde" ;	type chaîne ;
$Variable = "0.0";	type chaîne

b)tableaux :

PHP, comme il est le cas de plusieurs langages, permet l'utilisation des tableaux. Mais contrairement à des langages, il est possible de stocker des éléments de types différents dans un même tableau.

```
$Tableau[0] = 12;
$Tableau[1] = "exemple";
```

Avec PHP, il n'est pas nécessaire de préciser la valeur de l'index lorsque l'on veut remplir un tableau, car il assigne la valeur 0 au premier élément (si le tableau est vide) et incrémente les indices suivants.

```
$Tableau[] = 12;
$Tableau[] = " exemple";
```

On peut aussi parler de tableaux multidimensionnels qui peuvent être crées directement en utilisant plusieurs paires de crochets pour les index.

```
$Tableau[0][0] = 12;
$Tableau[0][1] = "CCM";
$Tableau[1][0] = 1245.652;
$Tableau[1][1] = "Au revoir";
```

PHP permet l'utilisation de chaînes de caractères au lieu de simples entiers pour définir les indices d'un tableau, on parle alors de *tableaux associatifs.*

```
$Toto["Age"] = 12;
$Toto["Adresse"] = "22 rue des bois fleuris";
$Toto["Nom"] = "NomFamille" ;
```

c) Les constantes

Avec PHP, les constantes sont définies grâce à la fonction *define()* ou avec const.

Syntaxe :

define("Nom_de_la_variable", Valeur);

```
<?php
define ("CONSTANT", "Bonjour à tous.");
echo CONSTANT; // affiche "Bonjour à tous."
echo Constant; // affiche "Constant" et une alerte
const X=1000 ;
echo X ;

define ("BONJOUR", "Salut toi.",TRUE);//true precise si le nom est insensible à la casse
echo BONJOUR; // affiche "Salut toi."
echo Bonjour; // affiche "Salut toi."

?>
```

3) les opérateurs

On distingue plusieurs types d'opérateurs :

- les opérateurs de calcul,
- les opérateurs d'assignation,
- les opérateurs d'incrémentation,
- les opérateurs de comparaison,
- les opérateurs logiques.

Ces opérateurs sont les mêmes qu'en langage C.

4) Les structures conditionnelles

a) L'instruction if

L'instruction if est la structure de test la plus basique. Elle permet d'exécuter une série d'instruction si jamais une condition est réalisée.

Syntaxe :

```
if (condition réalisée) {
                          bloc d'instructions
                        }
```

Remarques :

- La condition doit être entre des parenthèses,
- Il est possible de définir plusieurs conditions à remplir avec les opérateurs ET et OU (&& et ||),
- S'il n'y a qu'une instruction, les accolades ne sont pas nécessaires... ,
- L'expression *if … else* permet d'exécuter une autre série d'instruction en cas de non-réalisation de la condition.

Syntaxe :

```
if (condition réalisée) {
                          bloc d'instructions
                        }
else {
      autre bloc d'instructions
     }
```

Mais aussi on peut utiliser l'expression *if… elseif… else* pour enchaîner une série d'instructions et éviter d'avoir à imbriquer des instructions *if*.
Syntaxe :

```
if (condition réalisée) {
                        bloc d'instructions
                         }
elseif (autre condition réalisée) {
                                    autre bloc d'instructions
                                   }
else  {
      encore un autre bloc d'instructions
      }
```

b)L'instruction switch :

L'instruction *switch* permet de faire plusieurs tests de valeurs sur le contenu d'une même variable. Ce branchement conditionnel simplifie beaucoup le test de plusieurs valeurs d'une variable.

Syntaxe :

switch (*Variable*)
{
case *Valeur1*:
Liste d'instructions
break;
case *Valeur2*:
Liste d'instructions
break;
case *Valeurs...*:
Liste d'instructions
break;
default:
Liste d'instructions
break;
}

5) Les boucles

La façon la plus commune de faire une boucle, est de créer un compteur (une variable qui s'incrémente) et de faire arrêter la boucle lorsque le compteur dépasse une certaine valeur.

a) La boucle for

L'instruction *for* permet d'exécuter plusieurs fois la même série d'instructions

Syntaxe :

```
 for (compteur; condition; modification du compteur) {
                        bloc d'instructions
                         }
```

Par exemple:

```
for ($i=1; $i<6; $i++) {
   echo "$i<br>";
}
```

b) L'instruction while

L'instruction *while* représente un autre moyen d'exécuter plusieurs fois la même série d'instructions.

Syntaxe :

while (condition réalisée) {
bloc d'instructions
}

c) La boucle foreach

Bien sûr, l'idéal pour parcourir les valeurs d'un tableau, c'est une boucle.
La boucle foreach (pour chaque élément) présente l'avantage de parcourir la totalité d'un tableau, même si l'on n'a aucune idée du nombre d'éléments qu'il contient.

```
<?php
foreach(<tableau> as <element>)
{
<instructions>
}
```

```
<?php
//construction du tableau semaine
$semaine=array('lundi','mardi','mercredi', 'jeudi', 'vendredi', 'samedi', 'dimanche');
//parcours du tableau
foreach($semaine as $jour){
echo'- '.$jour.'<br/>';
}
/*
Pour chaque valeur du tableau $semaine, compose la variable $jour et affiche le jour puis va à la ligne...
*/
<?php
$membres = array('BrYs', 'mathieu', 'Yogui');
//parcours du tableau :
foreach($membres as $membre)
{
echo $membre; //affiche tour à tour "BrYs" puis "mathieu" puis "Yogui"
}
//parcours avec clefs :
foreach($membres as $i => $membre)
{
//affiche tour à tour "0 BrYs" puis "1 mathieu" puis "2 Yogui"
echo $i . ' ' . $membre;
}
//alternative :
foreach($membres as $i => $membre)
{
//affiche tour à tour "0 BrYs" puis "1 mathieu" puis "2 Yogui"
echo $i . ' ' . $membres[$i];
}
```

d) Saut inconditionnel

Il peut être nécessaire de faire sauter à la boucle une ou plusieurs valeurs sans pour autant mettre fin à celle-ci.
La syntaxe de cette expression est "*continue;*" (cette instruction se place dans une boucle!), on l'associe généralement à une structure conditionnelle pour ne pas exécuter les lignes situées entre cette instruction et la fin de la boucle.

Exemple: Imaginons que l'on veuille imprimer pour $x allant de 1 à 10 la valeur de 1/($x-7) ... il est évident que pour $x=7 il y aura une erreur. Heureusement, grâce à l'instruction *continue* il est possible de traiter cette valeur à part puis de continuer la boucle.

```
for ($x=1; $x<=10; $x++) {
   $a = $x-7;
   if ($a == 0) {
      echo "division par 0";
     continue;
   }
   echo "1/$a<br>";

}
```

e) Arrêt inconditionnel

Pour arrêter prématurément la boucle, pour une autre condition que celle précisé dans l'en-tête de la boucle. L'instruction *break* permet d'arrêter une boucle. Il s'agit, tout comme *continue*, de l'associer à une structure conditionnelle, sans laquelle la boucle ne ferait jamais plus d'un tour!

```
for ($x=1; $x<=10; $x++) {
   $a = $x-7;
   if ($a == 0) {
      echo "division par 0";
      break;
   }
   echo "1/$a<br>";

}
```

f) Arrêt d'exécution du script

PHP autorise l'utilisation de la commande *exit*, pour interrompre totalement l'interprétation du script, ce qui signifie que le serveur n'envoie plus d'informations au navigateur : le script est figé dans son état actuel. Cette instruction est particulièrement utile lors de l'apparition d'erreur !

C. Les fonctions

1) Déclaration d'une fonction :

La déclaration d'une fonction peut se faire n'importe où dans le code et ceci grâce au mot-clé *function.*

Syntaxe :

function Nom_De_La_Fonction(type1 argument1, type2 argument2, ...)
{
liste d'instructions
}

Remarques:

- le nom de la fonction suit les mêmes règles que les noms des variables.
- Les arguments sont facultatifs, mais s'il n'y a pas d'arguments, les parenthèses doivent rester présentes.
- Le type des arguments est facultatif.

2) Appel de fonction

Pour exécuter une fonction, il suffit de faire appel à elle.

Syntaxe :

Nom_De_La_Fonction();

3) Renvoi d'une valeur par une fonction

La fonction peut renvoyer une valeur grâce au mot-clé return. Lorsque l'instruction return est rencontrée, la fonction évalue la valeur qui la suit, puis la renvoie au programme appelant.

Syntaxe :

return valeur_ou_variable;

4) Les arguments d'une fonction

Il est possible de passer des arguments à une fonction. Ce passage d'arguments se fait au moyen d'une liste d'arguments (séparés par des virgules) entre parenthèses suivant immédiatement le nom de la fonction. A noter qu'il est possible de donner une valeur par défaut à ces arguments en faisant suivre le nom de la variable par le signe "=" puis la valeur que l'on affecte par défaut à la variable.

```
<?
function dire_texte($qui, $texte = 'Bonjour')
{
 if(empty($qui)){ // $qui est vide, on retourne faux
  return false;
 }else{
  echo "$texte $qui"; // on affiche le texte
  return true; // fonction exécutée avec succès
 }}
?>
```

Ainsi cette fonction peut être appelée de deux façons différentes:

```
<?
// Passage des deux paramètres
dire_texte("cher phpeur", "Bienvenue"); // affiche "Bienvenue cher phpeur"

// Utilisation de la valeur par défaut du deuxième paramètre

dire_texte("cher phpeur"); // affiche "Bonjour cher phpeur"
?>
```

5) Passage de paramètre par référence

Une autre méthode pour modifier une variable consiste à la faire précéder du caractère &, précisant qu'il s'agit alors d'un alias : la valeur de la variable est modifiée à la sortie de la fonction. On parle alors de passage par référence.

```
?
function dire_texte($qui, &$texte)
{
 $texte = "Bienvenue $qui";
}
$chaine = "Bonjour ";
dire_texte("cher phpeur",$chaine);
echo $chaine; // affiche "Bienvenue cher phpeur"
?>
```

6) Retourner plusieurs variables

Lorsque vous souhaitez qu'une fonction retourne plusieurs valeurs, le plus simple est d'utiliser un tableau.

```
<?
```

```
function nom_fonction()
{
.....
return array( $variable1, $variable2, $variable3 );
// on retourne les valeurs voulues dans un tableau
}
$retour = nom_fonction();
echo "$retour[0] - $retour[1] - $retour[2]";
?>
```

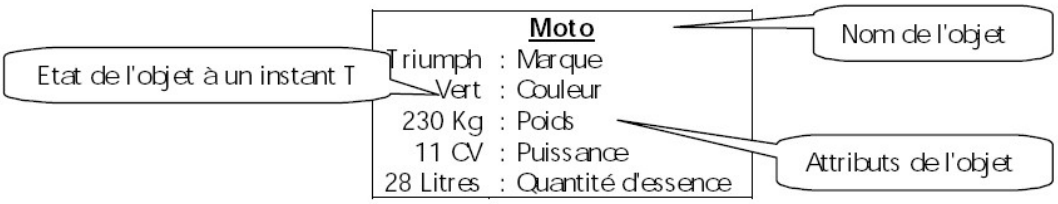
II. Les bases du POO en PHP

A. Les notions de la programmation orientée objet

Le PHP offre le concept de classes. Pour pouvoir manipuler des objets, il est essentiel de définir des classes, c'est-à-dire définir la structure d'un objet.
La P.O.O. (programmation orientée objet) possède de nombreuses vertus universellement reconnues désormais. Notamment, elle se fonde sur la programmation structurée, elle contribue à la fiabilité des logiciels et elle facilite la réutilisation de code existant. Elle introduit de nouveaux concepts, en particulier ceux d'objets, d'encapsulation, de classe d'héritage et de polymorphisme.

1) Le concept d'objet et de classe

L'informaticien et le non informaticien peuvent avoir un langage commun basé sur ce concept. Un objet au sens informatique du terme permet de désigner une représentation "abstraite" d'une chose concrète du monde réel ou virtuel. Un objet présente les 3 caractéristiques suivantes :
Objet = État + Comportement
L'état d'un objet définit la valeur des données (ou attributs), par exemple dans le cas d'un objet Moto, celui-ci pourrait être caractérisé par les attributs suivants : la marque, la couleur, le poids, la puissance, la quantité d'essence... Ce que l'on représente graphiquement par :



L'état de l'objet peut être amené à changer durant son cycle de vie. Par exemple, la quantité d'essence et le poids de la moto varient en permanence lorsque celle-ci roule.
Le concept de classe correspond simplement à la généralisation de la notion de type que l'on rencontre dans les langages classiques. En effet, une classe n'est rien d'autre que la description d'un ensemble d'objets ayant une structure de données commune et disposant des mêmes méthodes. Les objets encore appelé instance sont comme des variables d'un tel type classe. Bien entendu, seule la structure est commune, les valeurs des attributs ou champs étant propres à chaque objet. En revanche, les méthodes sont effectivement communes à l'ensemble des objets d'une même classe.

- **Les attributs** ou *données membres*: données représentant l'état de l'objet,
- **Les méthodes** ou *fonctions membres*: opérations applicables aux objets.

2) Le principe d'encapsulation

En programmation structurée, un programme est constitué de sous programmes (fonctions et procédures) et de structures de données généralement indépendants.

En P.O.O., un programme met en œuvre différents *objets*. Chaque objet associe des *données* et des *sous programmes* agissant exclusivement sur les données de l'objet. On parlera de *méthodes* et d'attributs pour désigner les sous programmes et les données.
En P.O.O., on réalise ce que l'on nomme une *encapsulation des données*.
Cela signifie qu'il n'est pas possible d'agir directement sur les données d'un objet sans passer par ses méthodes, qui jouent ainsi le rôle d'interface obligatoire. Ainsi l'appel d'une méthode est en fait l'envoi d'un message à l'objet.
L'intérêt du principe de l'encapsulation est que, vu de l'extérieur, un objet se caractérise uniquement par les spécifications de ses méthodes, la manière dont sont réellement implantées les données étant sans importance. Elle réalise donc une *abstraction des données.* L'encapsulation des données présente augmente manifestement la qualité de logiciel.
Elle facilite considérablement la maintenance : une modification éventuelle de la structure des données d'un objet n'a d'incidence que sur l'objet lui-même ; les utilisateurs de l'objet ne seront pas concernés par la teneur de cette modification. De la même manière, l'encapsulation des données facilite la réutilisation d'un objet.

3) L'héritage

Un autre concept important en P.O.O. est celui d'héritage. Il permet de définir une nouvelle classe à partir d'une classe existante, à laquelle on ajoute de nouvelles données et de nouvelles méthodes. La conception de la nouvelle classe, qui hérite des propriétés et des methodes, peut ainsi s'appuyer sur des réalisations antérieures parfaitement au point et les spécialiser à volonté. Comme on peut s'en douter, l'héritage facilite largement la réutilisation de produits existants.
Cette technique s'appliquera aussi bien aux classes que vous serez amenés à développer qu'aux très nombreuses classes fournies en standard avec Java.
Certains langages, tels C++, offrent la possibilité d'un héritage multiple : une même classe peut hériter simultanément de plusieurs autres. Ce n'est pas le cas de Java, mais nous verrons que la notion d'*interface* permet de traiter plus élégamment les situations correspondantes.

4) Le polymorphisme

En Java, comme généralement, en P.O.O., une classe peut *redéfinir* certaines des méthodes héritées de sa classe de base. Cette possibilité est la clé du *polymorphisme*, c'est-à-dire la possibilité de traiter de la même manière des objets de types différents, pour peu qu'ils soient issus de classes dérivées d'une même classe de base. Plus précisément, on utilise chaque objet comme s'il était de cette classe de base, mais son comportement effectif dépend de sa classe effective (dérivée de cette classe de base), en particulier de la manière dont ses propres méthodes ont été redéfinies.

B. Les classes et objets

1) Déclaration d'une classe

Avec Php, cette définition se fait de la manière suivante :

```
class Nom_de_la_classe {
   // Déclarations des données membres
   private $Donnee_Membre_1;
   private $Donnee_Membre_2;
   // Déclarations des méthodes
   function Nom_de_la_fonction_membre1(parametres) {
   //liste d'instructions;}
}
class Student {
// propriétés de la classe
```

```
private $name;
private $scores;
// fonction "constructeur" pour créer l'objet (voir explication plus loin)
function __construct ($nom) {
$this->name = $nom;
}
public function affiche(){
echo "etudiant ".$this->name. "a le score ".$this->scores.
}
// end of Student class definition
}
```

2) Instanciation de la classe

Après avoir déclaré une classe, il faut instancier des objets pour pouvoir l'exploiter. Cette opération se fait à l'aide du mot clé *new* permettant de faire des objets découlant d'une classe. La syntaxe du mot clé new est la suivante :
$Nom_de_l_objet = new Nom_de_la_classe();
A partir du moment où l'objet est instancié, il possède des propriétés qui lui sont propres.

3) Accéder aux propriétés d'un objet

L'accès aux propriétés d'un objet se fait grâce au nom de l'objet, suivi d'une flêche (->) représentée par un moins (-) et un signe supérieur (>), puis du nom de la donnée membre (sans le signe $). Par exemple :
$Nom_de_l_objet->Nom_de_la_donnee_membre = Valeur;

4) Accéder aux méthodes d'un objet

L'accès aux méthodes d'un objet se fait comme pour l'accès aux propriétés, c'est-à-dire par le nom de l'objet, suivi d'une flèche et du nom de la méthode. La méthode est suivie de parenthèses, contenant les paramètres, s'il y'en a. L'accès à une méthode se fait donc de la façon suivante :
$Nom_de_l_objet->Nom_de_la_fonction_membre(parametre1,parametre2,...);

```
// Ajouter quelques notes à Nathalie:
$nath->addScore(4); $nath->addScore(5); $nath->addScore(3);
$nath->addScore(6);
// Imprimer sa moyenne:
echo "Grade obtained by ".$nath->getName(). ": " .$nath->getGrade();
```

5) Constructeur et destructeurs

a) Constructeurs

Le constructeur est la fonction qui est appelée automatiquement lorsque vous créez une nouvelle instance d'une classe à l'aide de l'opérateur *new*.
__construct ([mixed args [, ...]])
PHP 5 permet aux développeurs de déclarer des constructeurs pour les classes. Les classes qui possèdent une méthode constructeur appellent cette méthode à chaque création d'une nouvelle instance de l'objet, ce qui est intéressant pour toutes les initialisations dont l'objet a besoin avant d'être utilisé.
Note : Les constructeurs parents ne sont pas appelés implicitement si la classe enfant définie un constructeur. Si vous voulez utiliser un constructeur parent, il sera nécessaire de faire appel à *parent::__construct()*.
Exemple d'utilisation des nouveaux constructeurs unifiés

```php
<?php
class BaseClass {
   function __construct() {
      print "In BaseClass constructor\n";
   }
}

class SubClass extends BaseClass {
   function __construct() {
      parent::__construct();
      print "In SubClass constructor\n";
   }
}

$obj = new BaseClass();
$obj = new SubClass();
?>
```

Pour des raisons de compatibilité ascendante, si PHP 5 ne peut pas trouver une fonction *__construct()* pour une classe donnée, il cherchera une fonction constructeur représentée, comme dans l'ancien style (PHP < 5), par le nom de la classe.

b) Destructeurs

__destruct ( )

PHP 5 introduit un concept de destructeur similaire aux autres langages orientés objet, comme le *C++*. La méthode destructeur doit être appelée aussitôt que toutes les références à un objet particulier sont effacées ou lorsque l'objet est explicitement détruit.

Exemple avec un Destructeur

```php
<?php
class MyDestructableClass {
          private $name;
   function __construct() {
      print "In constructor\n";
      $this->name = "MyDestructableClass";
   }
   function __destruct() {
      print "Destruction de " . $this->name."\n";
   }
}

$obj = new MyDestructableClass();
?>
```

Tout comme le constructeur, le destructeur parent n'est pas appelé implicitement par le moteur. Pour exécuter le destructeur parent, vous devez appeler explicitement la fonction parent::__destruct dans le corps du destructeur.

L'instruction this

this permet de référencer l'objet courant. Cette instruction est principalement utilisée lors de la définition des constructeurs et des méthodes d'une classe. Dans le cas de la définition d'un constructeur, elle permet de faire référence à la future instance. Dans le cas de la définition d'une méthode, elle permet de faire référence à l'instance à laquelle s'appliquera cette méthode.

6) Visibilité

La visibilité d'une propriété ou d'une méthode peut être définie en préfixant la déclaration avec un mot-clé : *public*, *protected* ou *private*. Les éléments déclarés publics (*public*) peuvent être utilisés par n'importe quelle partie du programme. L'accès aux éléments protégés (*protected*) est limité aux classes filles et à la classe qui a défini l'élément. L'accès aux éléments privés (*private*) est uniquement réservé à la classe qui les a définis.

c) Visibilité des membres

Les données membres doivent être définies comme publiques, protégées ou privées.
Exemple Déclaration des membres

```
<?php
/**
 * Définition de MyClass
 */
class MyClass
{
   public $public = 'Public';
   protected $protected = 'Protected';
   private $private = 'Private';

   function printHello()
   {
      echo $this->private;
      echo $this->protected;
      echo $this->private;
   }
}
$obj = new MyClass();
echo $obj->public; // Fonctionne
echo $obj->protected; // Erreur fatale
echo $obj->private; // Erreur fatale
$obj->printHello(); // Affiche Public, Protected et Private
/**
 * Définition de MyClass2
 */
class MyClass2 extends MyClass
{
// On peut redéclarer les éléments publics ou protégés, mais pas les privés
   protected $protected = 'Protected2';

   function printHello()
   {
     echo $this->public;
     echo $this->protected;
     echo $this->private;
   }
}

$obj2 = new MyClass2();
echo $obj->public; // Fonctionne
```

```
echo $obj2->private; // Indéfini
echo $obj2->protected; // Erreur fatale
$obj2->printHello(); // Affiche Public, Protected2 et non Private
?>
```

La méthode de déclaration de variable en PHP 4 avec le mot clé var n'est plus valide pour les objets en PHP 5. Pour des raisons de compatibilité, une variable déclarée ainsi aura automatiquement une visibilité publique et une erreur de niveau E_STRICT sera générée.

d) Visibilité des méthodes

Les méthodes des classes doivent être définies en tant que publiques, privées ou protégées. Les méthodes sans déclaration seront automatiquement définies comme étant publiques.

Exemple Déclaration d'une méthode

```
<?php
/*** Définition de MyClass*/
class MyClass
{
   // Les contructeurs doivent être publics
   public function __construct() { }

   // Déclaration d'une méthode publique
   public function MyPublic() { }

   // Déclaration d'une méthode protégée
   protected function MyProtected() { }

   // Déclaration d'une méthode privée
   private function MyPrivate() { }

   // Celle-ci sera publique
   function Foo()
   {
      $this->MyPublic();
      $this->MyProtected();
      $this->MyPrivate();
   }
}

$myclass = new MyClass;
$myclass->MyPublic(); // Fonctionne
$myclass->MyProtected(); // Erreur fatale
$myclass->MyPrivate(); // Erreur fatale
$myclass->Foo(); // Public, Protected et Private fonctionnent
/*** Définition de MyClass2 */
class MyClass2 extends MyClass
{
   // Celle-ci sera publique
   function Foo2()
   {
      $this->MyPublic();
      $this->MyProtected();
```

```
        $this->MyPrivate(); // Erreur fatale
    }
}

$myclass2 = new MyClass2;
$myclass2->MyPublic(); // Fonctionne
$myclass2->Foo2(); // Public et Protected fonctionnent, non pas Private
?>
```

7) Statique

Le fait de déclarer des attributs ou des méthodes comme statiques vous permet d'y accéder sans avoir besoin d'instancier la classe. Un attribut déclaré comme statique ne peut être accédé avec l'objet instancié d'une classe (bien qu'une méthode statique le peut). Les méthodes et les membres statiques ne peuvent être redéfinis dans les sous-classes.
La déclaration *static* doit être faite après la déclaration de visibilité. Pour des raisons de compatiblité avec PHP 4, si aucune déclaration de visibilité n'est utilisée, alors le membre ou la méthode sera traité comme s'il avait été déclaré comme *public*.
Comme les méthodes statiques sont appelables sans instance d'objet créée, la pseudo variable *$this* n'est pas disponible dans la méthode déclarée en tant que statique.
En fait, les appels de méthodes statiques sont résolus au moment de la compilation. Lorsque l'on utilise un nom de classe explicite, la méthode est déjà identifiée complètement et aucune notion d'héritage n'est appliquée. Si l'appel est effectué par le mot clé *self*, alors *self* est traduit en la classe courante, qui est la classe appartenant au code. Ici aussi, aucune notion d'héritage n'est appliquée.
On ne peut pas accéder à des propriétés statiques à travers l'objet en utilisant l'opérateur ->.

Exemple avec un membre statique

```
<?php
class Foo
{
    public static $my_static = 'foo';

    public function staticValue() {
        return self::$my_static;}
}
class Bar extends Foo
{

    public function fooStatic() {
        return parent::$my_static;
    }
}
print Foo::$my_static . "\n";

$foo = new Foo();
print $foo->staticValue() . "\n";
print $foo->my_static . "\n";      // propriété my_static non définie
// $foo::my_static n'est pas possible
print Bar::$my_static . "\n";
$bar = new Bar();
print $bar->fooStatic() . "\n";
?>
```

Exemple avec une méthode statique

```
<?php
class Foo
{
   public static function aStaticMethod() {
      // ...
   }}

Foo::aStaticMethod();
?>
```

8) Constantes de classe

Il est possible de définir des valeurs constantes à l'intérieur d'une classe, qui ne seront pas modifiables. Les constantes diffèrent des variables normales du fait qu'on n'utilise pas le symbole *$* pour les déclarer ou les utiliser. Tout comme pour les membres statiques, on ne peut pas accéder aux valeurs constantes depuis une instance de l'objet (en utilisant *$object::constant*).

Exemple 19-16. Définition et utilisation d'une constante de classe

```
<?php
class MyClass
{
  const constant = 'valeur constante';

  function showConstant() {
    echo  self::constant . "\n";
  }
}

echo MyClass::constant . "\n";

$class = new MyClass();
$class->showConstant();
// echo $class::constant;  n'est pas autorisé
?>
```

CHAPITRE 2 : Concepts avancés de POO

Ce chapitre va aborder certains concepts avancés de la POO comme l'héritage, le polymorphisme, les classes et méthodes abstraites et la notion d'interface.

I. L'héritage

1) Présentation

Souvent, on peut avoir besoin d'une classe avec des méthodes et fonctions similaires à une autre classe. En fait, il est bon de définir des classes génériques, qui pourront être réutilisées et adaptées à tous les projets. Pour faciliter cela, une classe peut être une extension ou une spécialisation d'une autre classe. La classe dérivée appelée classe fille, hérite alors de toutes les méthodes et attributs de la classe de base, appelée classe mère, mais peut définir ses propres méthodes et attributs, qui s'ajouteront. PHP Objet ne gère que l'héritage simple, l'héritage multiple n'est pas supporté. Les héritages se font avec le mot clé '*extends*'.

Exemple :

```
class Voiture{   // déclaration de la classe mère
private  $matricule;
private  $marque;
   function __construct($matricule, $marque){        // constructeur
      $this-> matricule = $matricule;
      $this-> marque = $marque;
   }
   function setMatricule ($matricule) {
      $this-> matricule = $matricule;
   }
   function setMarque($marque) {
      $this->marque = $marque;
   }
   function getMatricule() {
      return $this->matricule;
   }
   function getMarque() {
      return $this-> marque;
   }
public function affiche(){
echo ("la voiture ".$this->matricule. "est de marque ".$this->marque) ;
}
}
class VoitureDeLuxe extends Voiture {   // déclaration de la sous classe
   private $couleur;
   function __construct($matricule, $marque, $couleur) {        // constructeur
   }
   function setCouleur($couleur) {
      $this->couleur = $couleur;
   }
   function getCouleur() {
      return $this->couleur;
   }
}
```

La nouvelle classe **Voituredeluxe** hérite de tous les attributs et méthodes de la classe parente **Voiture** dont elle est une extension (**extends**). Il est possible de redéfinir les méthodes ou d'en déclarer de nouvelles, etc.

2) L'instruction parent

Il arrive que vous ayez à écrire du code qui fait référence aux attributs et méthodes des classes mères. C'est encore vrai puisque la classe fille est une spécialisation de la classe mère.

Au lieu d'utiliser le nom littéral de la classe de base dans le code, on peut utiliser le mot réservé *parent*, qui représente la classe mère. En faisant cela, on évite d'appeler le nom de la classe mère directement dans le code.

Exemple :

```
class VoitureDeLuxe extends Voiture {   // déclaration de la sous classe
   private $couleur;
   function __construct($matricule, $marque, $couleur) {        // constructeur
      parent:: __construct($matricule, $marque) ;
         $this->couleur = $couleur;
   }
   function setCouleur($couleur) {
      $this->couleur = $couleur;
   }
   function getCouleur() {
      return $this->couleur;
   }
}
```

```
<?php
class A {
  function example() {
    echo "Je suis A::example() et je fournis une fonctionnalité de base.<br />\n";
  }
}
class B extends A {
  function example() {
    echo "Je suis B::example() et je fournis une fonctionnalité supplémentaire .<br />\n";
    parent::example();
  }
}
$b = new B;
// Cette syntaxe va appeler B::example(), qui, à sont tour, va appeler A::e//xample().
$b->example();
?>
```

II. Le polymorphisme

Le polymorphisme permet à **des objets de différentes classes d'être traités de manière uniforme**, en répondant de manière cohérente aux mêmes messages ou appels de méthodes. Cela favorise la flexibilité et l'extensibilité du code en **permettant à différentes classes d'avoir des comportements spécifiques tout en partageant une interface commune**.

En PHP, le polymorphisme est souvent mis en œuvre en utilisant l'héritage et la redéfinition de méthodes dans les classes filles. Il s'agit de reprendre la définition d'une méthode définit dans la

classe mère, dans la classe fille pour le spécialiser ou l'adapter. Par exemple, supposons que nous ayons une classe parente Forme avec une méthode calculerSurface()

```
class Forme {
   public function calculerSurface() {
      // Implémentation par défaut pour le calcul de la surface
   }
}
```

Nous pouvons créer des classes enfants telles que Cercle et Rectangle qui héritent de la classe parente Forme et redéfinissent la méthode calculerSurface() avec leur propre implémentation spécifique :

```
 class Cercle extends Forme {
   private $rayon;

   public function __construct($rayon) {
      $this->rayon = $rayon;
   }
   public function calculerSurface() {
      // Calcul de la surface d'un cercle : π * rayon²
      $surface = 3.14 * pow($this->rayon, 2);
     return $surface ;
   }
}
class Rectangle extends Forme {
   private $longueur;
   private $largeur;

   public function __construct($longueur, $largeur) {
      $this->longueur = $longueur;
      $this->largeur = $largeur;
   }
   public function calculerSurface() {
      // Calcul de la surface d'un rectangle : longueur * largeur
      $surface = $this->longueur * $this->largeur;
      return $surface ;
   }
}
/*********************************************/
$cercle = new Cercle(5);
echo " La surface du cercle est : ". $cercle->calculerSurface();
// Cela affichera : La surface du cercle est : 78.539816339745

$rectangle = new Rectangle(3, 2);
echo " La surface du rectangle est : ". $rectangle->calculerSurface();
// Cela affichera : La surface du rectangle est : 6
```

III. Abstraction de classes

PHP introduit les classes et les méthodes abstraites. Une classe abstraite est une classe qu'on ne peut pas instancier. Il n'est donc pas autorisé de créer une instance d'une classe définie comme abstraite. Toutes les classes contenant au moins une méthode abstraite doivent également être abstraites.

Une méthode abstraite est une méthode sans définition. Pour définir une méthode abstraite, il faut simplement déclarer la signature de la méthode et ne fournir aucune implémentation.
Lorsqu'une classe hérite d'une classe abstraite, elle doit redéfinir (donner la déclaration) de toutes les méthodes abstraites héritées de la classe mère abstraite ; de plus, ces méthodes doivent être définies avec la même (ou plus faible) visibilité. Par exemple, si la méthode abstraite est définie comme protégée, l'implémentation de la fonction doit être définie en tant que protégée ou publique.

```
<?php
abstract class Utilisateur{
protected $user_name;
protected $user_region;
protected $prix_abo;
protected $user_pass;
public const ABONNEMENT = 15;
public function __destruct(){
//Du code à exécuter
}
abstract public function setPrixAbo();
public function getNom(){
echo $this->user_name;
}
public function getPrixAbo(){
echo $this->prix_abo;}
}
?>
```

Définissons une classe Admin et Abonne qui héritent de la classe Utilisateur.

```
<?php
class Admin extends Utilisateur{
protected static $ban;
public function __construct($n, $p, $r){
$this->user_name = strtoupper($n);
$this->user_pass = $p;
$this->user_region = $r;
}
public function setBan(...$b){
foreach($b as $banned){
self::$ban[] .= $banned;}
}
public function getBan(){
echo 'Utilisateurs bannis : ';
foreach(self::$ban as $valeur){
echo $valeur .', ';}
}
public function setPrixAbo(){
if($this->user_region === 'Sud'){
return $this->prix_abo = parent::ABONNEMENT / 6;
}else{
return $this->prix_abo = parent::ABONNEMENT / 3; }
}
}
```

```
?>
<?php
class Abonne extends Utilisateur{
public function __construct($n, $p, $r){
$this->user_name = $n;
$this->user_pass = $p;
$this->user_region = $r;}
public function setPrixAbo(){
if($this->user_region === 'Sud'){
return $this->prix_abo = parent::ABONNEMENT / 2;
}else{
return $this->prix_abo = parent::ABONNEMENT; }
}
}
?>
```

IV. Interfaces

1) Présentation et création

Les interfaces objet permettent de créer du code qui spécifie quelles méthodes et variables une classe peut implémenter, sans avoir à définir comment ces méthodes seront gérées.
Les interfaces sont définies en utilisant le mot clé ***interface***, de la même façon qu'une classe standard mais sans aucun contenu de méthode.
Toutes les méthodes déclarées dans une interface doivent être publiques.
On va pouvoir définir une interface de la même manière qu'une classe mais en utilisant cette fois-ci le mot clef interface à la place de class.

```
<?php
interface Utilisateur{
public const ABONNEMENT = 15;
public function getNom();
public function setPrixAbo();
public function getPrixAbo();}
?>
```

```
<?php
  interface Entretien{
    public function peindre($c);
    public function nettoyer(); }
?>
```

2) L'implémentation avec Implements

Pour implémenter une interface, l'opérateur *implements* est utilisé. Toutes les méthodes de l'interface doivent être implémentées dans la classe ; si ce n'est pas le cas, une erreur fatale sera émise.

Exemple d'interface

```
<?php
class Abonne implements Utilisateur{
protected $user_name;
protected $user_region;
protected $prix_abo;
protected $user_pass;
```

```
public function __construct($n, $p, $r){
$this->user_name = $n;
$this->user_pass = $p;
$this->user_region = $r;}
public function getNom(){
echo $this->user_name;
}
public function getPrixAbo(){
echo $this->prix_abo;}
public function setPrixAbo(){
if($this->user_region === 'Sud'){
return $this->prix_abo = Utilisateur::ABONNEMENT / 2;
}else{
return $this->prix_abo = Utilisateur::ABONNEMENT;}
}
}
?>
```

```
<?php
  class Vehicule implements Entretien{
    private $couleur;
    private $propre;
    public function peindre($c){
      $this->couleur=$c;    }
    public function nettoyer(){
      $this->propre=true; }
  }
?>
```

3) Implémenter plusieurs interfaces

A l'inverse de l'héritage en PHP, on peut implémenter plusieurs interfaces simultanément en les séparant par des points virgule :

```
<?php
  interface Int1{
    public function f1(); }
  Interface Int2{
    public function f2();  }
  class Classe implements Int1, Int2{
    public function f1(){
    }
    public function f2(){
    }
  }
?>
```

CHAPITRE 3 : Autoloading, namespaces et traits

I. Comment inclure les fichiers en PHP

Dans une programmation en PHP, sans l'inclusion de fichiers on serait obligé de copier-coller le même code sur toutes les pages qui nécessitent une instruction identique.
Comme pour les fonctions par exemple, **il se peut qu'on soit amené à utiliser les mêmes fonctions dans des fichiers différents**. Vous auriez donc à recopier la définition de chaque fonction en haut de chaque page. Cette méthode rendrait les mises à jour plus difficiles. En effet, ça vous obligerait à parcourir toutes les pages pour modifier les définitions des fonctions.

1) Inclure un fichier avec la fonction include

Supposons que l'on a quelques fonctions sur le calcul des prix. On en aura souvent besoin dans les pages de notre site. On va les regrouper dans un fichier prix.php et on va essayer de l'inclure dans un fichier. Afin d'éviter de trimbaler ces fonctions partout, **la fonction include nous servira d'inclusion de fichier**.
La fonction en question se remplace elle-même par le contenu du fichier spécifié.
Contenu de prix.php :

```
<?php
function difference($prix1, $prix2)
{
$resultat=$prix1-$prix2;
return $resultat;
}
function calculttc($prixht,$tva)
{
$prixttc=$prixht*(1+$tva/100);
}
function calculht($prixttc,$tva)
{
$prixht=($prixttc*100)/($tva+100);
}
function calculremise($pourcentage, $prix)
{
$remise=$prix*$pourcentage;
}
?>
```

Dans une page catalogue.php, on a par exemple le prix hors taxe au départ. Cependant, on a besoin d'afficher le prix TTC.
Contenu de catalogue.php :

```
<?php
//affichage et calcul prix TTC
include ("prix.php"); //inclusion du fichier prix
$prixht=100;
define(TVA, 19.6);
echo "prix ttc:".calculttc($prixht,TVA);
echo calculttc($prixht,TVA);
?>
```

Dans une autre page, facture.php par exemple, on a besoin des différents prix et de calculer aussi la remise.

Contenu de facture.php :

```
<?php
//affichage et calcul de tous les prix
include ("prix.php");
$prixht=100;
define(TVA, 19.6);
$prixttc= calculttc($prixht,TVA);
$remise = calculremise(10,$prixht);
$apayer= difference($prixttc, $remise);
echo "prix ht:".$prixht."<br>";
echo "prix ttc:".$prixttc."<br>";
echo "prix remise 10%:".$remise."<br>";
echo "prix à payer:".$apayer."<br>";
?>
```

On 'inclue' le contenu du fichier prix.php dans les autres fichiers, en l'appelant par la fonction 'include', et en lui passant en paramètre le nom du fichier à inclure (dans l'exemple : prix.php).

De même pour les classes, il est conseillé de créer une classe par fichier, et pour utiliser une classe dans une autre il faut l'inclure.

2) Inclure des fichiers avec la fonction Require

L'instruction require fonctionne de la même façon qu'include. Toutefois, il existe une différence entre ces deux méthodes. Lorsqu'on tente d'inclure un fichier avec include, si le fichier n'existe pas, PHP affichera une alerte (warning) et continuer l'exécution du script**.** A l'inverse, la fonction require génèrera une erreur fatale. Il interrompt votre script en cas d'erreur. En d'autres termes, utiliser require() si veut que le script soit interrompu en cas d'inclusion manquant.

Contenu du fichier haut.php :

```
<table width="500" border="1" bgcolor="#66CCCC">
 <tr>
  <td><?php echo " Menu1"; ?></td>
  <td><?php echo " Menu2"; ?></td>
  <td><?php echo " Menu3"; ?></td>
  <td><?php echo " Menu4"; ?></td>
  <td><?php echo " Menu5"; ?></td>
  <td><?php echo " Menu6"; ?></td>
 </tr>
</table>
```

Contenu du fichier bas.php :

```
<table width="500" border="1" bgcolor="#FF9900">
  <tr>  <td><?php echo "Pieds de page"; ?></td>  </tr>
</table>
```

Contenu du fichier page1.php :

```
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=iso-8859-1" />
<title>page</title>
</head>
<?php require("haut.php"); ?>
```

```
<table width="500" height="500" border="1" >
  <tr align="center">
 <td>
    <?php echo "contenu php"; ?>
   </td>
  </tr>
</table>
<?php require("bas.php"); ?>
</html>
```

Bien sûr, notre page a encore besoin d'une retouche. Néanmoins, la base est déjà là. Ce qui est intéressant c'est que là encore on a bien vu l'intérêt des inclusions de fichier.
En effet, le haut et le bas de page ne changent pas lors de la navigation. Toutes les pages n'ont peut-être pas les mêmes contenus, mais le haut et le bas de page restent les mêmes quel que soit la page.
Dans ce cas, il suffit d'inclure le fichier haut.php, et le fichier bas.php dans toutes les pages. S'il y a quelque chose à modifier dans le fichier haut.php, on aura juste à toucher un seul fichier, et ceci impactera l'ensemble des pages faisant l'include.
Par conséquent, structurer ainsi la page évitera certains problèmes.

3) Inclusion avec les fonctions require_once et include_once

Il faut noter bien qu'il y a aussi deux autres fonctions require_once et include_once qui peuvent être utiles.
Ce sont les mêmes que require et include mais la différence est qu'elles s'assurent que le fichier qu'on inclue ne l'a pas déjà été. **Il est conseillé de les utiliser lorsque vous voulez être sûrs que le fichier ne sera inclus qu'une seule fois**, notamment pour éviter les déclarations qui engendrent des erreurs. Un fichier rempli de fonctions, par exemple, ne peut être inclus qu'une seule fois (par exécution). Les fonctions étant déjà déclarées, elles ne peuvent l'être une seconde fois, et cela génèrerait une erreur.

II. Autoloading

1) La fonction __autoload

On conseille toujours de séparer les classes dans différents fichiers. Le problème c'est que l'on est obligé ensuite de faire beaucoup de require ou include pour charger nos différentes classes. Heureusement l'autoloading nous permet de remédier à ce problème en incluant les classes dès que l'on en a besoin.
Le principe de base est de créer une fonction **__autoload** qui permettra à PHP de savoir comment charger ou incluire nos classes.
L'autoloading est un concept en PHP qui permet de spécifier un répertoire ou des répertoires pour charger les classes. Cela est beaucoup plus avantageux que d'utiliser require ou include pour charger les classes. C'est également une exigence pour utiliser les packages Composer.

```
function __autoload($class_name){
   require('class/' . $class_name . '.php');
}
```

Le problème de cette méthode c'est qu'on ne peut pas avoir plusieurs fois la même fonction et on ne peut donc pas créer plusieurs autoloader.

2) Fonction spl_autoload_register()

A partir de la version 5 de PHP, une fonctionnalité qui permet le chargement dynamique des classes est apparue permettant ainsi de faciliter au développeur l'inclusion des fichiers dont il aura besoin.
La fonction **spl_autoload_register()** enregistre une fonction de notre choix dans la pile d'autoload. Cette pile est destinée à enregistrer des fonctions et les appeler, d'une manière implicite, quand on en

aura besoin. De cette manière, si on fait appel à une classe (instanciation, héritage ou appel statique d'un membre) l'autoload se charge d'appeler la fonction qui permet d'inclure la classe souhaitée.

Supposons que la fonction qui inclue les fichiers de classes est la **appel($classe)** :

```
<?php
  function appel($classe){
    include($classe.".php");
  }
?>
```

Pour enregistrer cette fonction dans la pile d'autoload on fait comme ceci:

```
<?php
  spl_autoload_register("appel");
?>
```

On passe le nom de la fonction en tant que chaîne de caractères en guise d'argument de la fonction **spl_autoload_resigter()**. Dans ce cas, le moindre code qui invoque une classe quelconque pousse le compilateur à charger une à une les différentes fonctions enregistrées dans la pile, y compris la fonction appel(), en leur passant comme paramètre le nom de la classe.

Exemple

Contenu de la page **String.php**:

```
<?php
  class String{
    private $str;
    private $length;
    public function __construct($chaine){
      $this->str=$chaine;
      $this->length=strlen($this->str);
    }
  }
?>
```

Contenu de la page **index.php** (qui est placée, dans cet exemple, dans le même repertoire que **String.php**):

```
<?php
  function appel($classe){
    include($classe.".php");
  }
  spl_autoload_register("appel");
  $str=new String("Bonjour");
?>
```

Au moment d'instancier la classe **String**, le fichier qui contient sa définition (String.php) est automatiquement inclus.

3) Créer une classe Autoloader

On peut aussi créer une classe pour le faire

```
/**
 * Class Autoloader
 */
class Autoloader{
   /**
    * Enregistre notre autoloader
    */
```

```
    static function register(){
        spl_autoload_register(array(__CLASS__, 'autoload'));
    }
    /**
     * Inclue le fichier correspondant à notre classe
     * @param $class string Le nom de la classe à charger
     */
    static function autoload($class){
        require  $class . '.php';
    }
}
```

On utilise ici une classe avec des méthodes statique pour une meilleur organisation. On peut alors lancer notre autoloader simplement.

```
require 'autoloader.php';
Autoloader::register();
```

III. Les Namespaces

La logique est de créer une classe pour chaque notion. Pour se faire on risque d'avoir des collisions alors que deux classes ne peuvent effectivement pas avoir le même nom pour donc palier à cela on a recours aux namespaces. Les namespaces permettent de définir un nom de package que l'on pourra ensuite charger de manière automatique grâce à un autoloading.
Les espaces de noms sont conçus pour résoudre deux problèmes que rencontrent les auteurs de bibliothèques et d'applications lors de la réutilisation d'éléments tels que des classes ou des bibliothèques de fonctions :

1. Collisions de noms entre le code crée, les classes, fonctions ou constantes internes de PHP, ou celle de bibliothèques tierces.
2. La capacité de faire des alias ou de raccourcir des noms extrêmement long pour aider à la résolution du premier problème, et améliorer la lisibilité du code.

Les espaces de noms PHP fournissent un moyen pour regrouper des classes, interfaces, fonctions ou constantes.
Il est avantageux de les utiliser pour les projets importants où l'on importe de nombreux fichiers **PHP** avec des classes et des fonctions qui pourraient porter le même nom. Le **namespace** nous aide à éviter les conflits.

1) Déclaration d'un espace de noms

Les espaces de noms sont déclarés avec le mot-clé namespace. Un fichier contenant un espace de noms doit déclarer l'espace au début du fichier, avant tout autre code.

```
<?php
namespace MonProjet;
const CONNEXION_OK = 1;
class Connexion { /* ... */ }
function connecte() { /* ... */ }
?>
```

Le seul élément autorisé avant la déclaration d'espace de noms est la commande declare, pour définir l'encodage du fichier source. De plus, aucun code non-PHP ne peut précéder la déclaration d'espace de noms, y compris des espaces :

Exemple #2 Erreur de déclaration d'un espace de noms

```
<html>
<?php
namespace MonProjet; // erreur fatale : l'espace de noms doit être le premier élément du script
?>
```

De plus, contrairement à d'autres structures PHP, le même espace de noms peut être défini dans plusieurs fichiers, ce qui permet de scinder le contenu d'un espace de noms sur plusieurs fichiers.

2) Accéder aux éléments d'un espace de noms

Pour utiliser des éléments d'un espace de noms en particulier, il va falloir que PHP sache à quel espace de noms on fait référence. Pour cela, des règles similaires à la recherche d'un fichier sur notre ordinateur sont utilisées en PHP.
Pour accéder à un élément d'un espace de noms, on peut déjà préciser un nom non qualifié, c'est-à-dire ne préciser que le nom de l'élément en question.
Si on précise le nom d'un élément sans qualificatif (c'est-à-dire sans préfixe) dans un espace de noms nommé, alors c'est l'élément de même nom dans l'espace nommé qui sera utilisé.

```php
<?php
    namespace Exemple{
        class Utilisateur{ /*Code de la classe*/ };
        const VILLE = 'Toulon';
        function bonjour() { echo 'Bonjour<br>'; };
        $cp = 83000;

        /*Tente d'appeler bonsoir() depuis l'espace de noms courant, puis
         *depuis l'espace global en cas d'échec*/
        bonsoir();
    }
    namespace{
        function bonsoir(){
            echo 'Bonsoir !';
        }
    }
?>
```

On peut également utiliser un nom qualifié pour utiliser un élément d'un espace de noms. Le nom qualifié correspond au nom de l'élément préfixé par son chemin d'accès à partir de l'endroit où il est appelé.
Si on utilise l'écriture sous\bonjour() depuis notre espace de noms Exemple, par exemple, on indique qu'on souhaite accéder à la fonction bonjour() située dans exemple\sous\bonjour().
Finalement, on peut encore préciser un nom absolu pour accéder à un élément d'un espace de noms. Un nom absolu correspond au chemin complet de l'élément, c'est-à-dire au nom de l'élément préfixé de tous les espaces et sous espaces et commençant avec un antislash.

```
<?php
namespace Exemple{
//On inclut notre espace de noms exemple\sous
include 'sousexemple.namespace.php';
class Utilisateur{ /*Code de la classe*/ };
const VILLE = 'Toulon';
function bonjour() { echo 'Bonjour<br>'; };
```

```
$cp = 83000;
bonjour();
sous\bonjour();
\exemple\sous\bonjour();
}
namespace{
//Code global
}
?>

<!DOCTYPE html>
<html>
<head>
<title>Cours PHP & MySQL</title>
<meta charset="utf-8">
<link rel="stylesheet" href="cours.css">
</head>
<body>
<h1>Titre principal</h1>
<?php
include 'exemple.namespace.php';
?>
<p>Un paragraphe</p>
</body>
</html>
```

3) La commande namespace et la constante __NAMESPACE__

La constante magique __NAMESPACE__ contient le nom de l'espace de noms courant sous forme de chaine de caractères. Dans le cas où on l'appelle depuis l'espace global, elle contient une chaine vide.

On va pouvoir utiliser cette constante pour récupérer le nom de l'espace de noms courant et accéder à ses éléments ou à d'autres éléments en partant de ce nom dans des situations où on n'a pas accès directement au nom.

La commande namespace va représenter l'espace de noms courant. C'est l'équivalent de l'opérateur self des classes vu précédemment mais pour les espaces de noms.

On va donc également pouvoir s'en servir pour accéder à des éléments d'un espace de noms en représentant cet espace à l'aide de la commande namespace.

4) Importation (des éléments) d'un espace de noms et création d'alias

Créer un « alias » en PHP correspond à définir un autre nom pour désigner un élément déjà existant dans notre script.

Pour importer un élément d'un espace de noms, nous allons utiliser l'instruction **use**.

Cette instruction, en important un élément d'un espace de noms, va également automatiquement créer un alias en utilisant le nom en soi de l'élément. On va cependant également pouvoir définir un alias personnalisé avec le mot clef as.

Les exemples ci-dessous illustrent comment importer et créer des alias des différents éléments d'un espace de noms.

```
<?php
namespace Exemple\Sous{
class Utilisateur{ /*Code de la classe*/ };
```

```
const VILLE = 'Lyon';
function bonjour() { echo 'Salut<br>'; }
function bonsoir() { echo 'Bonsoir<br>'; }
function bonne_nuit() { echo 'Bonne nuit<br>'; }
$cp = 69000;
}
?>
```

```
<?php
namespace Exemple{
//On inclut notre espace de noms exemple\sous
include 'sousexemple.namespace.php';
class Utilisateur{ /*Code de la classe*/ };
const VILLE = 'Toulon';
function bonjour() { echo 'Bonjour<br>'; };
$cp = 83000;
use Exemple\Sous; //Revient à écrire use Exemple\Sous as Sous
use Exemple\Sous\Utilisateur as Sousutil; //Alias d'une classe
use function Exemple\Sous\bonjour as bjr; //Alias d'une fonction
use const Exemple\Sous\VILLE; //Importation d'une constante
//Import et alias de plusieurs fonctions avec une instruction use
use function Exemple\Sous\{bonsoir as bns, bonne_nuit as nuit};
var_dump($obj = new Sousutil);
echo '<br>';
bjr();
echo VILLE;
echo '<br>';
bns();
nuit();
}
namespace{
function bonjour(){
echo 'Bonjour depuis l\'espace global';
}
}
?>
```

IV. Les traits

1) Principe des traits

Les traits ont été intégrés au PHP à partir de sa version 5.4. Ils permettent de minimiser le code en réutilisant des méthodes déjà déclarées sans être obligé d'hériter d'une classe entière.

Un trait ne peut être ni instancié comme une classe ni implémenté comme une interface. Il s'agit d'un bloc de code qui sera réutilisé par une classe.

2) Création

Pour créer un trait on déclare le mot clé **trait** suivi de son nom (capitalisé de préférence) puis des accolades qui contiennent le corps comme ceci:

```php
<?php
  trait Notification{
    public function afficher(){
      echo "Alerte signalée!";
    }
  }
?>
```

3) Utilisation des traits

Pour se servir du trait **Notification** dans une classe on procède comme ceci:

```php
<?php
  class Operation{
    use Notification;
  }
  $op=new Operation();
  $op->afficher(); // Affiche "Alerte Signalée!"
?>
```

Vous avez compris que le code du trait **Notification** a été réutilisé comme s'il était déclaré directement dans la classe **Operation**.

CHAPITRE 4 : Gestion des erreurs et exceptions

Les exceptions c'est la technique de la POO pour gérer les erreurs.
L'objectif est qu'une erreur puisse être décelée et traitée comme on le souhaite, et non pas par le système : on redéfinit en quelque sorte les fonctions système de traitement d'erreur.
Cela permettra aussi de traiter l'erreur là où elle apparaît ou ailleurs dans le programme.
Plus généralement, une gestion propre des erreurs est un objectif important pour une application.

I. Les exceptions

1) Déclenchement avec throw

On peut déclencher manuellement une exception à l'aide de l'instruction « **throw** ». Voici un code qui provoque toujours une exception :

```
try {
$erreur="Message d'erreur";
throw new Exception($erreur);
echo 'Jamais exécuté';
} catch (Exception $e) {
echo 'Exception: '.$e->getMessage().' à la ligne '.$e->getLine()."\n";
}
```

A l'aide de l'instruction « **throw** » nous avons provoqué l'émission d'une exception qui sera gérée par la classe « Exception ». Dans ce cas, PHP va rechercher le bloc « **catch** » correspondant au type d'objet créé pour gérer l'exception, ici «**catch(Exception $e)** »

```
<?php
function additionner($a, $b) {
if (!is_numeric($a) || !is_numeric($b))
throw new Exception('Les deux paramètres doivent être des nombres');
return $a + $b;
}
function programme(){
echo additionner(12, 3), '<br />';
echo additionner('azerty', 54), '<br />';
echo additionner(4, 8);
}
programme();
```

Pour cet exemple l'appel de la fonction programme() sans traitement de l'exception va entrainer une exception.

```
15
Fatal error:
Uncaught Exception:
Les deux paramètres doivent être des nombres
……
```

2) try ... catch

PHP inclut à partir de PHP5 le support des exceptions avec le bloc « **try ... catch** ». Ainsi on peut placer du code « à risque » ou un code « exceptionnel » qui peut produire une erreur dans un bloc « **try** », et gérer l'erreur, selon son type, dans des blocs « **catch** ». On peut utiliser plusieurs blocs « **catch** » suivant le type d'erreur généré. Attention, quand c'est possible on gérera les erreurs avec du code normal.

```
try {
... // Code à risque d'erreur
}
catch (Exception $e) {
... // Code réglant la situation exceptionnelle
}
```

Au sein du bloc « **try** », l'exécution d'une instruction erronée va déclencher l'émission d'un signal : une exception. Ce signal interrompt l'exécution du reste de tout le bloc. En fonction du signal, PHP exécutera le bloc « **catch** » approprié.
On remarque que dans le bloc « **catch** », PHP gère les exceptions à l'aide d'une classe spéciale appelée « **Exception** » qui est la classe de base de gestion des erreurs. Une exception est en fait définie par un objet de cette classe ou d'une classe dérivée. On peut donc créer ses propres gestionnaires d'exceptions. Comme on peut écrire plusieurs blocs « **catch** » pour gérer les exceptions suivant le type d'objet.
Reprenons l'exemple précédent en traitant l'exception

```
try{
programme();
}
catch (Exception $e){
echo $e->getMessage(), '<br />';
}
echo 'FIN';
```

Résultat

```
15
Les deux paramètres doivent être des nombres
FIN
```

L'erreur a été attrapée dans la fonction appelante « main » : elle remonte jusqu'au main à travers la fonction programme.
La fonction programme se déroule jusqu'à l'erreur : le 15 est affiché et rien d'autre. Le catch est traité. Puis la suite du programme.
La classe Exception fournit des méthodes pour accéder au message d'erreur (getMessage), à la pile des fonctions (getTrace), et d'autres méthodes.

3) classe Exception et classes personnalisées

La classe « **Exception** » qui est la classe de base de gestion des exceptions dispose des méthodes suivantes :

- **__construct($erreur,$code)** : le constructeur avec le message d'erreur et le code
- **getMessage()** : retourne le message d'erreur généré
- **getCode()** : Code erreur de l'exception
- **getLine()** : numéro de la ligne du script ayant provoqué l'exception
- **getFile()** : nom du script PHP ayant provoqué l'exception
- **__toString()** : chaîne formatée à afficher.

Les méthodes « **getXXXX()** » sont finales. On ne peut donc pas les redéfinir. Mais le constructeur et la méthode « **__toString** » peuvent être étendus. On peut donc créer sa propre classe dérivée.

```
class MonException extends Exception {
public function __construct($message="Erreur par défaut", $code = 0) {
parent::__construct($message, $code);
}
}
```

```
$a=1;
try {
switch($a) {
case 1: throw new MonException("Ouille !",$a);
break;
case 2: throw new Exception("Aie !",$a);
break;
}
}
catch(MonException $e) {
echo $e->getMessage();
}
catch (Exception $e) {
echo 'Exception: '.$e->getMessage().' ligne '.$e->getLine()."\n";
}
```

Si $a vaut 1, « **`MonException`** » sera envoyé, si $a vaut 2, ce sera «**`Exception`** ».

Exemple

```
class MonException extends Exception {
public function __toString() {
return $this->message;
}
}
function additionner($a, $b) {
if (!is_numeric($a) || !is_numeric($b))
throw new MonException('MonException : Les deux paramètres
doivent être des nombres');
return $a + $b;
}
try{
programme();
}
catch (MonException $e){
echo $e, '<br />';
}
```

4) Plusieurs Catch à la suite

On peut tenter d'attraper une exception spécialisée puis une exception plus générale si on n'a pas réussi. Il suffit d'écrire deux (ou plus) catch à la suite, du plus spécialisé au plus général.

```
try{
programme();
}
catch (MonException $e){
echo ' MonException : ', $e, '<br />';
}
catch (Exception $e){
echo ' Exception : ', $e->getMessage(), '<br />';
}
```

5) Finally : exécuter du code avant une erreur fatale

Si une exception n'est pas attrapée, il y aura une erreur fatale.

Le bloc « finally » qui vient après un try – catch, permet de définir un code à exécuter avant l'arrêt du programme, Ce bloc permet de faire des opérations pour s'assurer que le programme ne s'arrête pas de façon bizzare (nettoyage, fermeture de fichiers, déconnexion, etc.)
Le bloc « finally » peut aussi apparaître directement après un try, sans catch.

6) PHP : Erreur ou Exception ?

Il se peut que dans certains cas, PHP déclenche naturellement une exception suivant l'exécution d'une fonction. Mais dans la plupart des cas, c'est l'ancien système de gestion des erreurs qui est utilisé. Ainsi le code suivant

```
try {
$a=fopen('toto','r');
}
catch(Exception $e) {
echo 'Exception: '.$e->getMessage();
}
```

ne donnera pas du tout le résultat escompté. L'exécution de « fopen » ne provoquera pas l'émission d'une exception mais une erreur PHP de type WARNING :

> **Warning**: fopen(toto) [function.fopen]: failed to open stream: No such file or directory in **C:\apachefriends\xampp\htdocs\objet\panier.php** on line **21**

PHP gère plusieurs niveaux d'erreurs : **WARNING**, **NOTICE**, etc. Par exemple, un problème d'ouverture de fichier émet une erreur de type WARNING, une erreur de syntaxe PARSE, l'utilisation d'une variable non déclarée un NOTICE. On peut régler et modifier les niveaux d'erreurs dans le php.ini, mais aussi dynamiquement via des fonctions.
Parmi ces fonctions, deux peuvent nous intéresser. « **set_error_handler** » permet de modifier le comportement par défaut de PHP lorsqu'il rencontre une erreur dans votre script. Au lieu d'exécuter son propre gestionnaire, il exécutera le vôtre. Attention, ce gestionnaire ne peut pas gérer certains types d'erreurs, mais dans le cas d'un simple **WARNING** ou **NOTICE** il n'y a pas de problème. On lui passe en paramètre le nom de sa propre fonction qui doit gérer l'erreur. Les paramètres sont assez parlants. Dans notre fonction, nous allons déclencher une exception à la réception de n'importe quelle erreur.

```
function gest_erreur($errno, $errstr, $errfile, $errline)
{
throw new Exception("$errstr line $errline",$errno);
}
set_error_handler("gest_erreur");
```

A l'exécution de notre bloc **« try...catch** », nous aurons cette fois
Exception: fopen(toto) [function.fopen]: failed to open stream: No such file or directory line 21 ce que nous voulions : nous avons bien déclenché une exception.

7) Exception par défaut

Si certaines instructions PHP génèrent des exceptions et que ces instructions ne sont pas au sein d'un bloc « **try...catch** », PHP générera une exception par défaut avec son propre gestionnaire d'exceptions. On peut modifier cet état avec la fonction « **set_exception_handler** ».

```
function exception_handler($exception) {
echo "Exception imprévue : " , $exception->getMessage(), "\n";
}
set_exception_handler('exception_handler');
```

CHAPITRE 5 : PHP POO et base de données (PDO)

I. Présentation

MySQL est une base de données implémentant le langage de requête SQL un langage relationnel très connu. Cette partie suppose connue les principes des bases de données relationnelles.
Il existe un outil libre et gratuit développé par la communauté des programmeurs libres : phpMyAdmin qui permet l'administration aisée des bases de données MySQL avec php. Cependant PHP peut se connecter aux autres bases de données comme Oracle, PostreSql, SQL Server …

Avec l'évolution des SGBD et des framework, il devient très important de faire la persistance des données par de nouvelles méthodes dont les ORM(Object Relationnel Mapping) pour faciliter l'échange entre les programmes objet et les bases de données relationnelles.
Le terme CRUD est étroitement lié avec la gestion des données numériques. Plus précisément, CRUD est un acronyme des noms des quatre opérations de base de la gestion de la persistance des données et applications :

CRUD-Operation	**SQL**
Create	INSERT
Read	SELECT
Update	UPDATE
Delete	DELETE

Plus simplement, le terme CRUD résume les fonctions qu'un utilisateur a besoin d'utiliser pour créer et gérer des données.

II. L'extension mysqli

L'extension mysqli fournit 2 interfaces. Elle supporte la programmation procédurale mais aussi, la programmation orientée objet.

A. La fonction mysqli

Les utilisateurs migrants depuis l'ancienne extension mysql préfèreront l'interface procédurale. Cette interface est similaire à celle utilisée par l'ancienne extension mysql. Dans la plupart des cas, les noms de fonctions ne diffèrent que par leurs préfixes. Quelques fonctions mysqli prennent un gestionnaire de connexion comme premier argument, alors que la fonction correspondante de l'ancienne interface mysql le prenait comme argument optionnel en dernière position.

1) Se connecter à la base de données

Pour se connecter à MySQL, on va utiliser cette fonction :

```
<?php
$bdd = mysqli_connect('serveur', 'utilisateur', 'mot_de_passe', 'base');
?>
```

- **$bdd** correspond à une variable où seront stockées les informations de la base de données. On s'en servira souvent.
- **serveur** correspond au serveur SQL.
- **utilisateur** correspond au nom d'utilisateur pour se connecter au serveur SQL.
- **mot_de_passe** correspond au mot de passe pour le serveur SQL !
- **base** correspond à votre base de données du serveur SQL.

On peut tester si la connexion a réussi ou non :

```
<?php
if($bdd = mysqli_connect('localhost', 'root', '', 'base'))
{
    // Si la connexion a réussi, rien ne se passe.
}
```

```
else // Mais si elle rate…
{
    echo 'Erreur'; // On affiche un message d'erreur.
}
?>
```

Dans cet exemple, MySQLi se connecte au serveur local.
Une fois que la connexion est établie, nous pouvons passer à nos premières requêtes...

2) Les requêtes simples

Nous allons apprendre à gérer les requêtes simples.
Ce sont les requêtes qui ne sont pas préparées, car ces dernières sont un peu plus complexes.

a) Le code de la requête

Le code de la requête est le suivant :

```
<?php
$resultat = mysqli_query($bdd, 'requete');
?>
```

- **$resultat** est la variable où seront stockées les données.
- **$bdd** est la variable retournée par la fonction de connexion à la base de données, voir plus haut.
- **'requete'** est une requête SQL.

Exemple :

```
<?php
$resultat = mysqli_query($bdd, 'SELECT * FROM membres LIMIT 0, 10');
?>
```

b) Afficher le résultat d'une requête simple

Pour afficher le résultat de ce type de requêtes, il suffit de *fetcher* dans une boucle *while*.

```
<?php
while($donnees = mysqli_fetch_assoc($resultat))
{
   echo $donnees['id'];
   echo "\n";
   echo $donnees['pseudo'];
}
?>
```

Que se passe-t-il ? En réalité, c'est relativement simple : *$donnees* est transformé en un tableau avec les noms des colonnes demandées dans la requête. Cette transformation est réalisée grâce à la fonction *mysqli_fetch_assoc($resultat)*, *$resultat* étant le résultat brut de la requête.

```
<?php
while($donnees = mysqli_fetch_assoc($resultat))
{
   echo $donnees['id'];
   echo "\n";
   echo $donnees['pseudo'];
}
mysqli_free_result($resultat);
?>
```

Eh oui, il ne faut pas oublier de fermer le curseur avec la fonction suivante :

```
<?php
```

```
mysqli_free_result($resultat);
?>
```

Fermer le curseur permet de libérer les données récupérées lors d'une requête SQL.

c) Les fonctions supplémentaires

```
mysqli_commit()   Valide la transaction courante
mysqli_next_result()    Prépare le prochain résultat d'une requête multiple
mysqli_select_db()   Sélectionne une base de données par défaut pour les requêtes
mysqli_stmt_affected_rows()   Le nombre total de lignes modifiées, effacées ou insérées
par la dernière
mysqli_num_fields()  Le nombre de champs dans un résultat
mysqli_num_rows()    Le nombre de lignes dans un résultat
mysqli_fetch_array()   Retourne une ligne de résultat sous la forme d'un tableau indexé,
mysqli_fetch_object()    Retourne la ligne courante d'un jeu de résultat sous forme
d'objet
mysqli_fetch_row() Récupère une ligne de résultat sous forme de tableau indexé
mysqli_free_result()   Libère la mémoire associée à un résultat
```

3) Les requêtes préparées

Pour travailler avec les requêtes préparées, il y a un certain ordre à suivre :

1. préparer la requête ;
2. lier les variables à la requête ;
3. exécuter la requête ;
4. si la requête renvoie un résultat, on continue : lier le résultat à des variables ;
5. *fetcher* le résultat.

Quasiment toutes les fonctions concernant les requêtes préparées commenceront par *mysqli_stmt_*.

a) Préparer la requête

On prépare la requête avec cette fonction :

```
<?php
$req_pre = mysqli_prepare($bdd, 'SELECT * FROM membres WHERE id = ?');
?>
```

$req_pre est une variable qui contient la requête préparée.

b) Lier les variables à la requête préparée

On les lie avec cette fonction un peu plus complexe :

```
<?php
mysqli_stmt_bind_param($req_pre, "i", $id);
?>
```

Cette fonction est en effet un peu particulière.
$req_pre est la variable de la requête préparée. i est le type de la variable qui suit. La variable correspond à ce qui va remplacer le point d'interrogation dans la requête.
Concernant les types de variables possibles, voici un petit tableau :

Lettre	Correspond à
i	un nombre entier
d	un nombre décimal
s	une chaîne de caractères

Ces lettres doivent être placées selon l'ordre dans lequel apparaissent les variables après, tel que les variables doivent apparaître dans l'ordre des points d'interrogation dans la requête. Les lettres doivent être dans le même paramètre.
Un exemple:

```
<?php
```

```
$req_pre = mysqli_prepare($bdd, 'INSERT INTO membres (pseudo, age, poids) VALUES ( ?, ?,
?)');
mysqli_stmt_bind_param($req_pre, "sid", $pseudo, $age, $poids);
?>
```

Disons que *$pseudo* vaut *quelquun*, que *$age* vaut *23* et que *$poid* vaut *52,357* (par exemple, hein).

c) Exécuter la requête

```
<?php
mysqli_stmt_execute($req_pre);
?>
```

Si la requête ne renvoie pas de résultat (les requêtes d'insertion, de mise à jour, de suppression), on s'arrête là. Sinon, il faut lier le résultat de l'exécution à des variables.

d) Lier le résultat à des variables

On peut lier le résultat d'une requête préparée à un tableau dont chacun aura le nom de la colonne du résultat. Vous allez vite comprendre :

```
<?php
mysqli_stmt_bind_result($req_pre, $donnees['pseudo'], $donnees['age'],
$donnees['email']);
?>
```

Voilà, si vous choisissez de tout sélectionner dans la requête, vous aurez une variable par colonne de la base. Si vous avez sélectionné les colonnes que vous souhaitez obtenir dans la requête, il faudra une variable par colonne demandée.

e) Fetcher le résultat

Si vous savez qu'il n'y aura qu'un résultat, il faut *fetcher* de cette façon :

```
<?php
mysqli_stmt_fetch($req_pre);
?>
```

Et seulement ensuite vous pourrez utiliser le tableau *$donnees*.
S'il y aura plusieurs résultats, il faut faire une boucle :

```
<?php
while(mysqli_stmt_fetch($req_pre))
{
      echo $donnees['pseudo'] . ", " . $donnees['age'] . " ans," . $donnees['email'];
}
?>
```

Exemples

Avec une requête qui ne renvoie pas de résultat

```
<?php
// include de la connexion SQL.

$req_pre = mysqli_prepare($bdd, 'INSERT INTO membres (pseudo, age, email) VALUES (?, ?,
?)');
mysqli_stmt_bind_param($req_pre, "sis", $pseudo, $age, $email);
mysqli_stmt_execute($req_pre);
```

```
?>
```

Avec une requête qui renvoie des résultats

```
<?php
// include de la connexion SQL.

$req_pre = mysqli_prepare($bdd, 'SELECT * FROM membres WHERE age = ? LIMIT 0, ?');
mysqli_stmt_bind_param($req_pre, "ii", $age, $nombre_resultat);
mysqli_stmt_execute($req_pre);
mysqli_stmt_bind_result($req_pre, $donnees['pseudo'], $donnees['age'],
$donnees['email']);
while(mysqli_stmt_fetch($req_pre))
{
    echo $donnees['pseudo'] . ", " . $donnees['age'] . " ans, " . $donnees['email'];
}
?>
```

Dans l'exemple ci-dessus, *pseudo*, *age* et *email* sont les seules colonnes de ma table.
Si j'avais demandé dans la requête SQL seulement *pseudo* et *email* j'aurais lié le résultat seulement à *$donnees['pseudo']* et *$donnees['email']*, dans l'ordre que je les ai indiqués dans la requête.

B. Avec la classe mysqli

1) Ouvrir une connexion la base de données MySQL

Avant d'exploiter les données qui se trouvent sur notre base, il nous faut ouvrir un canal de communication qui ne sera fermé qu'une fois toutes nos opérations effectuées.
Notre base de données se trouve sur un serveur désigné avec un nom de domaine, un adresse IP ou un alias. Pour nous y connecter, il nous faut un login et un mot de passe.
Pour se connecter, on utilise la classe MySQL comme ci-dessous :

```
<?php
$serveur = "mysql.monserveursql.com";
$base = "mabase";
$user = "monuser";
$pass = "monpass";

/*
$mysqli est une nouvelle instance de la classe mysqli
prédéfinie dans php et hérite donc de ses propriétés et méthodes
connexion à la base de données
*/
$mysqli = new mysqli($serveur, $user, $pass, $base);
// si la connexion se fait en UTF-8, sinon ne rien indiquer
$mysqli->set_charset("utf8");
/*
utilisation de la méthode connect_error
qui renvoie un message d'erreur si la connexion échoue
*/
if ($mysqli->connect_error) {
   die('Erreur de connexion ('.$mysqli->connect_errno.')'. $mysqli->connect_error);
}
?>
```

Ici, nous effectuons un test pour savoir si la connexion est effective ou pas avec l'alias *die()* (de *exit()*) : en cas d'échec, le code s'interrompra ici en affichant l'erreur rencontrée par MySQL (donnée dans la méthode *connect_error*.
On pourrait également faire un test plus classique comme ci-dessous :

```
<?php
if ($mysqli->connect_error) {
   echo 'connexion impossible... :'.$mysqli->connect_error;
}
else {
   echo 'connexion réussie : '.$mysqli->host_info;
}
?>
```

Afin d'éviter de définir les identifiants de connexion à chaque fois que qu'on exploite la base de données, il est utile de les préciser dans un fichier qui sera appelé à chaque fois via l'instruction *include()*. Ainsi, vous pourrez facilement les mettre à jour si besoin et renforcerez la sécurité de votre site web (en protégeant l'accès à ce fichier via un fichier .htaccess sur les serveurs Apache).

2) Fermer la connexion

Normalement, la fermeture de la connexion se fait automatiquement, mais afin de s'en assurer, on peut utiliser la méthode close()

```
<?php
$mysqli->close();
?>
```

3) Les methodes supplémentaires

- mysqli::$affected_rows — Retourne le nombre de lignes affectées par la dernière opération MySQL
- mysqli::change_user — Change l'utilisateur de la connexion spécifiée
- mysqli::close — Ferme une connexion
- mysqli::commit — Valide la transaction courante
- mysqli::$connect_errno — Retourne le code d'erreur de la connexion MySQL
- mysqli::$connect_error — Retourne le message d'erreur de connexion MySQL
- mysqli::__construct — Ouvre une connexion à un serveur MySQL
- mysqli::$errno — Retourne le dernier code d'erreur produit
- mysqli::$field_count — Retourne le nombre de colonnes pour la dernière requête
- mysqli::$info — Retourne des informations à propos de la dernière requête exécutée
- mysqli::$insert_id — Retourne l'identifiant automatiquement généré utilisé par la dernière requête
- mysqli::prepare — Prépare une requête SQL pour l'exécution
- mysqli::query — Exécute une requête sur la base de données
- mysqli::real_connect — Ouvre une connexion à un serveur MySQL
- mysqli::refresh — Rafraîchie
- mysqli::release_savepoint — Supprime le point de sauvegardé nommé du jeu des points de sauvegarde de la transaction courante
- mysqli::rollback — Annule la transaction courante

- mysqli::select_db — Sélectionne une base de données par défaut pour les requêtes

III. PDO : Interface d'accès aux bases de données

1) Présentation

PDO (***P**HP **D**ata **O**bjects*) vise à permettre la création d'un code comportant des accès aux bases de données en faisant abstraction du moteur de Système de Gestion de Bases de Données utilisé.
C'est une interface qui, à ce titre, permet aux codeurs l'utilisant de travailler sans avoir à trop se poser de questions par la suite... Enfin ceci dans l'idéal, car il y a un problème.
Cette difficulté concerne le respect des standards. Chaque SGBD a sa façon d'interpréter les standards et chaque SGBD dans sa façon de voir les choses essaye plus ou moins de s'y conformer, sans parler de tout ce qui peut être ajouté aux standards. Finalement, avec ces différents points, on se retrouve avec des possibilités différentes de requêtes pour une même action.
Concrètement pour le développeur, ceci engendre le fait que selon le SGBD, certaines requêtes devront être réécrites spécifiquement.
Mais s'il est vrai que certaines requêtes devront être réécrites spécifiquement, beaucoup (de par le mécanisme qui sous-tend PDO) resteront valables.

2) Avantages de PDO

On peut citer les avantages suivants :

- *interface pour SGBD* : plus besoin de s'occuper de savoir quelle SGBD est derrière (en théorie) ;
- *orienté objet* : les objets PDO et PDOStatement peuvent être étendus, il est donc tout à fait possible de personnaliser et remodeler une partie du comportement initial ;
- *exception* : les objets de l'interface PDO utilisent des exceptions, il est donc tout à fait possible d'intégrer facilement un système de gestion des erreurs.

3) Activation - Php Windows

Pour activer le PDO il faut le faire dans le '*php.ini*' , rendez-vous dans la partie où il y a la liste des extensions. Faites une recherche sur '*; Windows Extensions*', vous devriez y arriver directement.
Bien : maintenant, vérifiez que vous avez la ligne suivante :
```
;extension=php_pdo.dll
```
Si c'est le cas, décommentez la ligne afin d'avoir :
```
extension=php_pdo.dll
```
Sinon, rajoutez la ligne à la main.
PDO est activé mais cela n'est pas suffisant : maintenant, il faut activer le *driver* correspondant au type de BDD que vous souhaitez utiliser.
Par exemple pour MySQL, rajoutez pour la ligne suivante (ou activez-la si elle est présente) :
```
extension=php_pdo_mysql.dll
```
Enregistrez, relancez le serveur.
Pour savoir si votre SGBD peut être utilisé et activé, allez dans le répertoire d'extensions de PHP '*Ext*' et regardez les fichiers portant l'extension *.dll* et commençant par : *php_pdo_**.
Si vous le trouvez, il ne vous reste plus qu'à activer l'extension.
Ainsi, si vous ne voulez pas MySQL :
```
extension=php_pdo_mysql.dll
```
mais plutôt Oracle :
```
extension=php_pdo_odbc.dll
```
Vous pouvez trouver le tableau de correspondance des DLL avec le SGBD sur la documentation officielle (https://www.php.net/manual/fr/pdo.drivers.php).

4) Connexion : création et gestion d'erreur

Après avoir activé les extensions nécessaires au bon fonctionnement de PDO on peut se connecter aux bases de données.
Tous les exemples qui seront traités à partir d'ici utiliseront MySQL comme SGBD cependant par la suite, rien n'empêche d'utiliser autre chose, la forme sera toujours la même (seuls les paramètres de connexion peuvent varier d'un SGBD à l'autre, tout comme les requêtes).
Création d'une connexion :

```
<?php
$PARAM_hote='localhost'; // le chemin vers le serveur
$PARAM_port='3306';
$PARAM_nom_bd='sdz'; // le nom de votre base de données
$PARAM_utilisateur='root'; // nom d'utilisateur pour se connecter
$PARAM_mot_passe=''; // mot de passe de l'utilisateur pour se connecter
$connexion = new
PDO('mysql:host='.$PARAM_hote.';port='.$PARAM_port.';dbname='.$PARAM_nom_bd,
$PARAM_utilisateur, $PARAM_mot_passe);
?>
```

Si vous tentez une connexion via un code similaire à celui-ci, et qu'une erreur se produit, vous verrez que l'erreur affichée expose le DSN. Il est nécessaire d'utiliser un gestionnaire d'erreur, même simple. Ceci n'est pas difficile, et présenté juste à la suite.

Comment gérer les erreurs ?

Vous pouvez avoir une erreur pour plusieurs raisons ; vous avez donné de mauvais paramètres, ou vous avez spécifié des paramètres qui ne correspondent pas au *driver* que vous souhaitez utiliser.
Afin de savoir d'où vient le problème, le mieux est encore d'afficher 'proprement' les erreurs par la gestion des exceptions

```
<?php
try
{
	$connexion = new PDO('mysql:host='.$PARAM_hote.';dbname='.$PARAM_nom_bd,
$PARAM_utilisateur, $PARAM_mot_passe);
}

catch(Exception $e)
{
	echo 'Erreur : '.$e->getMessage().'<br />';
	echo 'N° : '.$e->getCode();
}
?>
```

Le catch est chargé d'intercepter une éventuelle erreur apparue dans le try :

```
<?php
try
{
	$connexion = new PDO('mysql:host='.$PARAM_hote.';dbname='.$PARAM_nom_bd,
$PARAM_utilisateur, $PARAM_mot_passe);
}

catch(Exception $e)
{
	echo 'Une erreur est survenue !';
```

```
    die();
}
?>
```

5) Méthodes : exec et query

PDO fait la distinction entre deux familles de requêtes. Il est possible sur une BDD de récupérer des informations, mais aussi d'effectuer des changements (qui peuvent prendre la forme d'ajout, suppression ou modification).
Si c'est un SELECT donc récupérer un résultat, on doit alors utiliser 'query' ; si c'est pour apporter des changements (INSERT, UPDATE, DELETE) à la BDD, on doit utiliser 'exec'.
Ces exemples étant fictifs, vous devrez les adapter avec vos propres tables. :)

Utilisation de la méthode : exec

Supposons que nous souhaitons modifier le mot de passe des membres :

```
<?php
$connexion = new PDO("mysql:host=$PARAM_hote;dbname=$PARAM_nom_bd", $PARAM_utilisateur, $PARAM_mot_passe); // connexion à la BDD

$connexion->exec("UPDATE membres SET mot_pass='toto'"); // on modifie le //mot de passe de tous les utilisateurs
?>
```

A ce stade, la modification a bien eu lieu mais ceci peut ne pas être suffisant.
Imaginons que nous souhaitons savoir le nombre de lignes qui ont été affectées par ce changement, nous aurons alors ceci :

```
<?php
// création d'une connexion

$nombre_changement=$connexion->exec("UPDATE membres SET mot_pass='toto'"); // on modifie le mot de passe de tous les utilisateurs
echo "La requête à modifié : $nombre_changement lignes.";
?>
```

Utilisation de la méthode : query

Imaginons que nous souhaitons connaître tous les membres :

```
<?php
$connexion = new PDO("mysql:host=$PARAM_hote;dbname=$PARAM_nom_bd", $PARAM_utilisateur, $PARAM_mot_passe); // connexion à la BDD

$resultats=$connexion->query("SELECT membre FROM membres ORDER BY membre ASC"); // on va chercher tous les membres de la table qu'on trie par ordre //croissant
$resultats->setFetchMode(PDO::FETCH_OBJ); // on dit qu'on veut que le //résultat soit récupérable sous forme d'objet
while( $ligne = $resultats->fetch() ) // on récupère la liste des membres
{
  echo 'Utilisateur : '.$ligne->membre.'<br />'; // on affiche les membres
}
$resultats->closeCursor(); // on ferme le curseur des résultats
?>
```

Comme vous le voyez, ça ressemble sensiblement à ce qui se fait habituellement, avec cependant deux détails que je me dois de soulever.
Le premier point concerne le 'setFetchMode' qui permet d'indiquer sous quel format on souhaite récupérer les résultats de la requête précédente. Il existe plusieurs méthodes qui sont les suivantes :

PDO::FETCH_ASSOC, PDO::FETCH_BOTH, PDO::FETCH_OBJ, etc.,
Toujours concernant le mode, sachez qu'il existe une méthode plus courte d'écriture pour spécifier le mode, en le spécifiant directement dans fetch(), ce qui donne :

```
<?php
$ligne = $resultats->fetch(PDO::FETCH_OBJ)
?>
```

Le second point de remarque concerne closeCursor : il permet de fermer le curseur associé à un jeu de résultats, il doit donc être fermé une fois que vous avez fini de récupérer les résultats, et également être fermé avant toute autre requête.
Il vous est donc fortement conseillé de le mettre, sinon vous risquez des erreurs.
A remarquer qu'en cas d'erreur de la requête, le résultat ne sera pas un objet mais un booléen valant false (cette erreur étant alors soit due à une mauvaise requête, soit à un curseur qui n'a pas été fermé).
En effet pour réaliser une requête SELECT afin de récupérer des données de la BDD il faut utiliser "query" et pour les trois autres (INSERT, DELETE et UPDATE) il faut utiliser "exec" comme dans les quatres exemples ci-dessous qui respectivement insère, met à jour, efface et recherche un login et mot de passe d'un membre:

INSERT

```
$connexion->exec("INSERT INTO membres(champ_login,champ_mdp)
VALUES('login','mot_de_passe')");
```

On insert une entrée dans la BDD
UPDATE

```
$connexion->exec("UPDATE membres SET champ_login='login',champ_mdp='mot_de_passe'
WHERE champ_id_membre = id_membre");
```

On met a jour une entrée dans la BDD
DELETE

```
$connexion->exec("DELETE FROM membres WHERE champ_id_membre='id_membre'");
```

On supprime une entree dans la BDD
SELECT

```
$resultats=$connexion->query("SELECT membre FROM membres WHERE
champ_id_membre='id_membre'");
$resultats->setFetchMode(PDO::FETCH_OBJ);
while( $resultat = $resultats->fetch() )
{
      echo 'Utilisateur : '.$resultat->membre.'<br>';
}
$resultats->closeCursor();
```

On recherche une entrée dans la BDD
Aller plus loin avec SELECT :
Il est possible de raccourcir la syntaxe de la requete en combinant ces deux lignes :

```
$resultats->setFetchMode(PDO::FETCH_OBJ);
while( $resultat = $resultats->fetch() )
```

Par :

```
while( $resultat = $resultats->fetch(PDO::FETCH_OBJ) )
```

6) Méthode : prepare

Cette méthode prépare une requête SQL à être exécutée en offrant la possibilité de mettre des marqueurs qui seront substitués lors de l'exécution.
Il existe deux types de marqueurs qui sont respectivement **?** et les marqueurs nominatifs. Ces marqueurs ne sont pas mélangeables : donc pour une même requête, il faut choisir l'une ou l'autre des options.

Avantages de cette méthode :

- optimisation des performances pour des requêtes appelées plusieurs fois ;
- protection des injections SQL (plus besoin de le faire manuellement).

Exemple de cette méthode sans marqueur :
Imaginons que nous voulons récupérer la liste des membres.

```
<?php
// ouverture d'une connexion ...

$requete_prepare_1=$connexion->prepare("SELECT identifiant FROM membres"); // on prépare notre requête
$requete_prepare_1->execute();
while($lignes=$requete_prepare_1->fetch(PDO::FETCH_OBJ))
{
	echo $lignes->identifiant.'<br />';
}
?>
```

Exemple de cette méthode avec marqueur nominatif :
Imaginons maintenant que nous voulons un identifiant bien précis.

```
<?php
// ouverture d'une connexion ...

$requete_prepare_1=$connexion->prepare("SELECT identifiant FROM membres WHERE ID_membre = :id"); // on prépare notre requête
$requete_prepare_1->execute(array( 'id' => 1 ));
$lignes=$requete_prepare_1->fetch(PDO::FETCH_OBJ);
echo $lignes->identifiant.'<br />';
?>
```

Il est possible d'avoir plus de marqueurs : dans ce cas, il faut les rajouter dans la requête et dans execute, le tableau contiendra autant de clés et de valeurs qu'il y a de marqueurs dans la requête préparée.
Exemple de cette méthode avec marqueur **?** :
Nous voulons toujours notre identifiant unique.

```
<?php
// ouverture d'une connexion ...

$requete_prepare_1=$connexion->prepare("SELECT identifiant FROM membres WHERE ID_membre = ? "); // on prépare notre requête
$requete_prepare_1->execute(array( 1 ));
$lignes=$requete_prepare_1->fetch(PDO::FETCH_OBJ);
echo $lignes->identifiant.'<br />';
?>
```

7) Sécuriser sans passer par des requêtes préparées

Comme nous l'avons vu précédemment, il est possible de préparer une requête, ce qui va la sécuriser de facto.
Cependant certains usages (soit par facilité, soit par certaines limitations imposées par des requêtes) nécessitent de passer par des requêtes non préparées, dans ce cas, on est en droit de vouloir que la requête soit sécurisée.
Pour faire cela il existe la méthode *quote*. Cette méthode s'utilise directement avec la ressource de connexion.
Exemple :

```
<?php
$RessourceDeConnection = new PDO(...); // ouverture d'une connection
$RessourceDeConnection->query("SELECT id_membre FROM membres WHERE nom =
".$RessourceDeConnection->quote($nom, PDO::PARAM_STR));
?>
```

Ici nous ouvrons une nouvelle connexion, ensuite nous faisons une requête de type SELECT avec la méthode query prévue à cet effet.
Remarquez que pour appeler le nom, nous utilisons directement la ressource de connexion à laquelle nous ajoutons la méthode *quote*, en premier argument on met la valeur qui devra être protégée et en deuxième argument (qui est optionnel), on met le type de valeur à protéger.
Valeurs possibles pour le deuxième argument :
- **PDO::PARAM_STR** : pour une chaîne de caractères ;
- **PDO::PARAM_INT** : pour le type 'integer' de SQL ;
- **PDO::PARAM_NULL** : pour le type NULL de SQL ;
- **PDO::PARAM_BOOL** : pour un booléen ;
- **PDO::PARAM_LOB** : pour le type 'objet large' de SQL.

Par défaut, si vous ne spécifiez pas de deuxième argument, la valeur PDO::PARAM_STR sera utilisée.

IV. ORM et CRUD

1) Présentation

Le **mapping objet-relationnel**, ou **ORM** (Object-Relational Mapping), est une technique clé utilisée en programmation pour établir une connexion entre les **objets** du code et les **données** contenues dans des bases de données **relationnelles**. En simplifiant les interactions entre le code orienté objet et les structures de données, l'ORM permet aux développeurs de manipuler les données de manière plus intuitive, sans avoir à se plonger dans les complexités du langage de requête **SQL**. Cette approche utilise des descripteurs de **métadonnées** pour créer une couche d'abstraction qui facilite l'accès et la gestion des informations tout en améliorant la **productivité** des développeurs.
Le **mapping objet-relationnel** (ORM) est une technique qui facilite l'interaction entre le code des applications orientées objet et les bases de données relationnelles. En utilisant des descripteurs de **métadonnées**, cette approche crée une couche abstraite qui simplifie la gestion des données sans nécessiter une connaissance approfondie des langages de requête tels que le SQL. Cet article explore les principes fondamentaux de l'ORM, son fonctionnement, ses avantages et ses inconvénients.

a) Définition de ORM

Le mapping objet-relationnel est un mécanisme qui permet aux développeurs d'accéder et de manipuler des objets directement dans le code tout en s'occupant de la persistance des données dans une **base de données relationnelle**. Cet interface élimine donc la nécessité de créer manuellement des instructions SQL pour interagir avec la base de données, ce qui accroît l'efficacité du développement.

b) Comment fonctionne l'ORM ?

Le système ORM utilise un processus de **transformation d'informations** : il interprète et traduit les états des objets et crée un **schéma structuré** qui établit la relation entre les objets et les tables de données. Ainsi, lorsqu'une application modifie un objet de données, l'ORM se charge d'effectuer les opérations nécessaires (insertion, mise à jour ou suppression de données) dans la base de données relationnelle, sans que le développeur ait à gérer le code SQL directement.

c) Avantages de l'utilisation d'un ORM

Les ORM présentent plusieurs avantages. Tout d'abord, ils permettent aux développeurs de maintenir une vue cohérente des objets au fil du temps, même lorsque les sources de données changent. De plus, l'ORM simplifie considérablement le besoin de requêtes SQL, rendant ainsi le développement plus rapide. Ces outils permettent également une meilleure gestion des **schémas de base de données** et offrent la possibilité d'intégrer de nouvelles technologies sans modifications destructrices du code.

d) Inconvénients de l'ORM

Malgré leurs avantages, les ORM ne sont pas sans inconvénients. Ils peuvent entrainer des problèmes de performance en raison de la surcharge de code générée pour l'abstraction. D'un autre côté, une mauvaise configuration de l'ORM peut mener à des bases de données mal conçues, rendant leur maintenance difficile. En outre, des erreurs de correspondance entre les tables de données et les objets peuvent également causer des problèmes d'applications difficiles à diagnostiquer.

e) Comparaison entre ORM et SQL

Alors que l'utilisation de requêtes SQL peut offrir plus de flexibilité et de détails, elle impose également une plus grande responsabilité sur les développeurs en ce qui concerne la sécurisation du code. Les ORM, quant à eux, fournissent un niveau d'abstraction qui réduit ce fardeau, facilitant ainsi la sécurité des bases de données. La décision d'utiliser l'ORM ou le SQL dépend donc du contexte du projet et des préférences des développeurs.

f) Qu'est-ce que le CRUD ?

CRUD est un acronyme pour : créer , lire , mettre à jour et supprimer (Create, Read, Update et Delete). Les opérations CRUD sont une manipulation de données de base pour la base de données.

2) Design Pattern "Active Record"

Le design pattern

Active Record est un modèle architectural utilisé en génie logiciel, popularisé par Martin Fowler dans son livre *Patterns of Enterprise Application Architecture*. Il s'agit d'une approche pour accéder aux données d'une base de données relationnelle en encapsulant la logique d'accès au sein des objets du domaine.

Concept Clé

Dans le modèle Active Record :

- **Chaque table** de la base de données est mappée à une **classe** dans l'application.
- **Chaque ligne (ou tuple)** de la table est représentée par une **instance (un objet)** de cette classe.
- L'objet contient à la fois les **données** (ses attributs correspondent aux colonnes de la table) et le **comportement** (méthodes pour interagir avec la base de données, telles que save(), update(), delete(), et les requêtes de recherche).

Exemple (pseudo-code)

Pour une table users avec les colonnes name et age, une classe User pourrait être utilisée comme suit :

```
pseudo-code
// Créer un nouvel enregistrement
$user = new User();
$user->name = "John Doe";
$user->age = 25;
$user->save(); // Exécute une requête INSERT

// Lire un enregistrement
$user = User::find_by_name('John Doe'); // Exécute une requête SELECT

// Mettre à jour un enregistrement
$user->name = 'Dave';
$user->save(); // Exécute une requête UPDATE

// Supprimer un enregistrement
$user->destroy(); // Exécute une requête DELETE
Avantages et Inconvénients
```

a) Avantages

- **Simplicité et rapidité de développement** pour les applications CRUD simples.
- **Facile à comprendre** et à prendre en main pour les débutants.
- **Moins de code d'infrastructure** à écrire manuellement.

b) Inconvénients

- **Violation du principe de responsabilité unique** (Single Responsibility Principle), car les objets mélangent logique métier et logique d'accès aux données.
- **Difficultés pour les tests unitaires**, car les objets Active Record sont étroitement couplés à la base de données, nécessitant une connexion DB pour les tests
- **Limites avec des modèles de domaine complexes** (héritage, relations N-N complexes).

c) Utilisation dans les Frameworks

Le pattern Active Record est très répandu, notamment dans les frameworks web :

- Ruby on Rails utilise une implémentation célèbre appelée "Active Record".
- Laravel utilise Eloquent ORM, qui est basé sur ce modèle.
- D'autres ORM, comme TypeORM, proposent de choisir entre Active Record et le pattern Data Mapper, qui est son alternative principale

3) Design Pattern “Data Mapper”

Le **Data Mapper** est un modèle de conception (design pattern) architectural qui crée une couche d'abstraction entre la **logique métier (domaine)** de l'application et la **couche de persistance (base de données)**. Son objectif principal est de permettre à ces deux couches d'évoluer indépendamment l'une de l'autre.

d) Concept et Fonctionnement

Le modèle Data Mapper repose sur une classe ou un ensemble de classes dédiées (les "mappers" ou "Data Access Objects - DAO") qui gèrent le transfert bidirectionnel des données.

- **Objets Métier (Domaine Objects/Entités) :** Ce sont de simples objets (POJO en Java, classes simples en PHP) qui contiennent les données et la logique métier, sans aucune

connaissance des détails de la base de données (comme les requêtes SQL, les noms de tables ou les colonnes).
- **Data Mapper :** C'est la couche intermédiaire responsable de toutes les opérations CRUD (Create, Read, Update, Delete). Il traduit les données de la base de données en objets métier en mémoire, et vice-versa.
- **Base de Données :** Le système de stockage persistant des données.

Contrairement au modèle **Active Record**, où les objets métier gèrent eux-mêmes leur propre persistance (par exemple, un objet Utilisateur aurait une méthode save()), le Data Mapper sépare complètement ces responsabilités.
Avantages et Inconvénients

e) Avantages

- **Séparation des préoccupations (Single Responsibility Principle) :** La logique métier et la logique de persistance sont clairement séparées, ce qui rend le code plus propre et plus facile à maintenir.
- **Flexibilité :** Permet de modifier le schéma de la base de données ou même de changer de type de base de données (par exemple, passer de MySQL à PostgreSQL) sans affecter la logique métier de l'application.
- **Testabilité :** Les objets métier peuvent être testés indépendamment de la base de données, en utilisant des mappers simulés (mocks).
- **Adaptabilité :** Idéal pour les applications complexes où le modèle objet ne correspond pas directement au schéma relationnel de la base de données.

f) Inconvénients

- **Complexité accrue :** Introduit plus de classes et une couche d'abstraction supplémentaire, ce qui peut sembler excessif pour de petites applications.
- **Surcharge initiale :** La mise en place manuelle du modèle demande plus de travail de codage au début du projet. Les ORM modernes comme Doctrine 2 utilisent ce modèle pour automatiser une grande partie de ce travail.